

Lecture8a

October 31, 2025

1 CS5489 - Machine Learning

2 Lecture 8a - Unsupervised Learning: Dimensionality Reduction

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Supervised Learning

- *Supervised learning* considers input-output pairs $(\mathbf{x}, y)$
 - learn a mapping from input to output.
 - *classification*: output $y \in \pm 1$
 - *regression*: output $y \in \mathbb{R}$
- “Supervised” here means that the algorithm is learning the mapping that we want.

4 Unsupervised Learning

- Unsupervised learning only considers the input data $\mathbf{x}$.
 - There are no output values.
- **Goal:** Try to discover inherent properties in the data.
Clustering

Dimensionality Reduction

Manifold Embedding

5 Outline

1. Linear Dimensionality Reduction for Vectors
2. Linear Dimensionality Reduction for Text
3. Non-linear Dimensionality Reduction
4. Manifold Embedding

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
```

```
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from mpl_toolkits.mplot3d import Axes3D
```

6 Dimensionality Reduction

- **Goal:** Transform high-dimensional vectors into low-dimensional vectors.
 - Dimensions in the low-dim data represent co-occurring features in high-dim data.
 - Dimensions in the low-dim data may have semantic meaning.
- **For example:** document analysis
 - high-dim: bag-of-word vectors of documents
 - low-dim: each dimension represents similarity to a topic.
- **Example:** image analysis
 - approximate an image as a weighted combination of several basis images
 - represent the image as the weights.

7 Reasons for Dimensionality Reduction

- Preprocessing - make the dataset easier to use
- Reduce computational cost of running machine learning algorithms
- Remove noise - convert to lower dimension, and then project back to high-dimension
- Make the results easier to understand (visualization)

8 Linear Dimensionality Reduction

- Project the original data onto a lower-dimensional hyperplane (e.g., line, plane).
 - I.e, Move and rotate the coordinate axis of the data
- Represent the data with coordinates in the new component space.
- Equivalently, approximate the data point $\mathbf{x}$ as a linear combination of basis vectors (components) in the original space.
 - original data point $\mathbf{x} \in \mathbb{R}^d$
 - approximation: $\hat{\mathbf{x}} = \sum_{j=1}^p w_j \mathbf{v}_j$
 - * $\mathbf{v}_j \in \mathbb{R}^d$ is a basis vector and $w_j \in \mathbb{R}$ the corresponding weight.
 - the data point $\mathbf{x}$ is then represented its corresponding weights
 - * $\mathbf{w} = [w_1, \dots, w_p] \in \mathbb{R}^p$
- Several methods for linear dimensionality reduction.
- **Differences:**
 - goal (reconstruction vs classification)
 - unsupervised vs. supervised
 - constraints on the basis vectors and the weights.
 - reconstruction error criteria

9 Principal Component Analysis (PCA)

- Unsupervised method
- **Goal:** preserve the variance of the data as much as possible
 - choose basis vectors along the maximum variance (longest extent) of the data.
 - the basis vectors are called *principal components* (PC).

```
[2]: ## Visualization
↳#####
def plot_ellipse(ax, musigma, color="k", lw=1):
    """
    Based on
    http://stackoverflow.com/questions/17952171/
    ↳not-sure-how-to-fit-data-with-a-gaussian-python.
    """

    mu, sigma = musigma

    # Compute eigenvalues and associated eigenvectors
    vals, vecs = linalg.eigh(sigma)

    # Compute "tilt" of ellipse using first eigenvector
    x, y = vecs[:, 0]
    theta = degrees(arctan2(y, x))

    # Eigenvalues give length of ellipse along each eigenvector
    # plot 2 stdevs
    w, h = 2 * sqrt(vals)

    #ax.tick_params(axis='both', which='major', labelsize=20)
    ellipse = matplotlib.patches.Ellipse(mu, w, h, angle=theta, fill=False,
    ↳color=color, lw=lw) # color="k")
    ellipse.set_clip_box(ax.bbox)
    ellipse.set_alpha(1.0)
    ax.add_artist(ellipse)
```

```
[3]: def plot_basis(X, vo, Xp=None, sp=None, showleg=True, Y=None, axbox1=None,
    ↳axbox2=None,
        recon=False, showellipse=False, legfont=9):
    if sp is None:
        sp = ((1,2,1), (1,2,2))
        cols = ['k', 'g']

    axh = [None, None]

    if Y is None:
        Y = zeros(len(X))
```

```

    mycmap = plt.get_cmap('bwr')
else:
    mycmap = rbow

Ls = []
LL = []

# normalize v
v = vo.copy()
numd = v.shape[0]
for j in range(numd):
    v[j,:] /= linalg.norm(v[j,:])
    # flip direction if necessary
    if all(v[j,:] < 0):
        v[j,:] *= -1.0

Xm = mean(X,axis=0) # get mean
if Xp is None:
    Xp = dot(X-Xm,v.T) # project data onto v

axh[0] = plt.subplot(sp[0][0], sp[0][1], sp[0][2])
# plot points
if recon: # , edgecolor='none'
    XX = dot(Xp, v) + Xm
    plt.scatter(X[:,0], X[:,1], s=5, c=Y, cmap=mycmap, edgecolors='k',
↪alpha=0.5)
    plt.scatter(XX[:,0], XX[:,1], s=5, c='r', edgecolors='k', )
else:
    plt.scatter(X[:,0], X[:,1], s=5, c=Y, cmap=mycmap, edgecolors='k')
if showellipse:
    uY = unique(Y)
    for myY in uY:
        myX = X[Y==myY,:]
        myCov = cov(myX, rowvar=False)
        myMu = mean(myX, axis=0)
        myColor = (myY - amin(Y)) / (amax(Y)-amin(Y))
        plot_ellipse(plt.gca(), (myMu, 4*myCov), color=mycmap(myColor))
        tmp = plt.scatter(myMu[0], myMu[1], s=160, marker='X',
↪edgecolor='k', color=mycmap(myColor), label='$\mu_{}$'.format(myY+1))
        Ls.append(tmp)
        LL.append('$\mu_{}$'.format(myY+1))

if axbox1 is not None:
    plt.axis(axbox1)
else:
    plt.axis('equal')
plt.grid(True)

```

```

if recon:
    plt.title('Original space ( $\hat{x}$ )')
else:
    plt.title('Original space (x)')

tmp = plt.xlim()
hw = (tmp[1]-tmp[0])/30
lw = hw/10

mind = zeros(numd)
maxd = zeros(numd)
for j in range(numd):
    mind[j] = Xp[:,j].min()
    maxd[j] = Xp[:,j].max()

    ofact = 0.9
    Xm2 = Xm + v[j]*maxd[j]*ofact
    Xm1 = Xm + v[j]*mind[j]*ofact
    vl = v[j]*(maxd[j]-mind[j])*ofact

    D = {'width':lw, 'head_width':hw,
         'color':cols[j], 'length_includes_head':True}
    if recon:
        D['alpha'] = 0.2
    tmp = plt.arrow(Xm1[0], Xm1[1], vl[0], vl[1], **D)
    Ls.append(tmp)
    LL.append("PC "+str(j))

plt.plot(Xm[0], Xm[1], 'ks')
if showleg:
    plt.legend(Ls, LL, loc='lower right', fontsize=legfont)

axh[1] = plt.subplot(sp[1][0], sp[1][1], sp[1][2])
plt.arrow(mind[0]*ofact, 0, (maxd[0]-mind[0])*ofact, 0, width=lw,
↪head_width=hw, color=cols[0])
if numd == 2:
    plt.arrow(0, mind[1]*ofact, 0, (maxd[1]-mind[1])*ofact, width=lw,
↪head_width=hw, color=cols[1])
    plt.scatter(Xp[:,0], Xp[:,1], s=10, marker='+', c=Y, cmap=mycmap) #,
↪edgecolors='k'
    plt.xlabel('w1')
    plt.ylabel('w2')
else:
    plt.scatter(Xp[:,0], zeros(len(Xp)), s=10, marker='x', c=Y,
↪cmap=mycmap) #edgecolors='k'
    plt.xlabel('w')
    if showellipse:

```

```

        for myY in uY:
            myX = Xp[Y==myY,:]
            myCov = diag([var(myX), 0.001])
            myMu = array([mean(myX), 0])
            myColor = (myY - amin(Y)) / (amax(Y)-amin(Y))
            plot_ellipse(plt.gca(), (myMu, 4*myCov), color=mycmap(myColor))
            plt.scatter(myMu[0], myMu[1], s=160, marker='X', edgecolor='k',
↳ color=mycmap(myColor), label='$\mu_{%d}$'.format(myY+1))
            plt.legend(fontsize=legfont)

    if axbox2 is not None:
        plt.axis(axbox2)
    else:
        plt.axis('equal')
    plt.grid(True)
    plt.title('Low-dim space (w)')

    return axh

```

```

<>:47: SyntaxWarning: invalid escape sequence '\m'
<>:49: SyntaxWarning: invalid escape sequence '\m'
<>:57: SyntaxWarning: invalid escape sequence '\h'
<>:47: SyntaxWarning: invalid escape sequence '\m'
<>:49: SyntaxWarning: invalid escape sequence '\m'
<>:57: SyntaxWarning: invalid escape sequence '\h'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2259380869.py:4
7: SyntaxWarning: invalid escape sequence '\m'
    tmp = plt.scatter(myMu[0], myMu[1], s=160, marker='X', edgecolor='k',
color=mycmap(myColor), label='$\mu_{%d}$'.format(myY+1))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2259380869.py:4
9: SyntaxWarning: invalid escape sequence '\m'
    LL.append('$\mu_{%d}$'.format(myY+1))
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2259380869.py:5
7: SyntaxWarning: invalid escape sequence '\h'
    plt.title('Original space ($\hat{x}$)')

```

```

[4]: # image showing blobs of dots and various directions of largest variance
random.seed(5230)
Xblob,Y = datasets.make_blobs(100,2,centers=[[2,1]],cluster_std=1)
tmp = random.normal(size=(2,2))+3.5*eye(2)
Xblob = dot(Xblob,tmp)

axbox1 = [-5, 20, -10, 15]
axbox2 = [-15, 15, -15, 15]

vfig = plt.figure(figsize=(10,6))

```

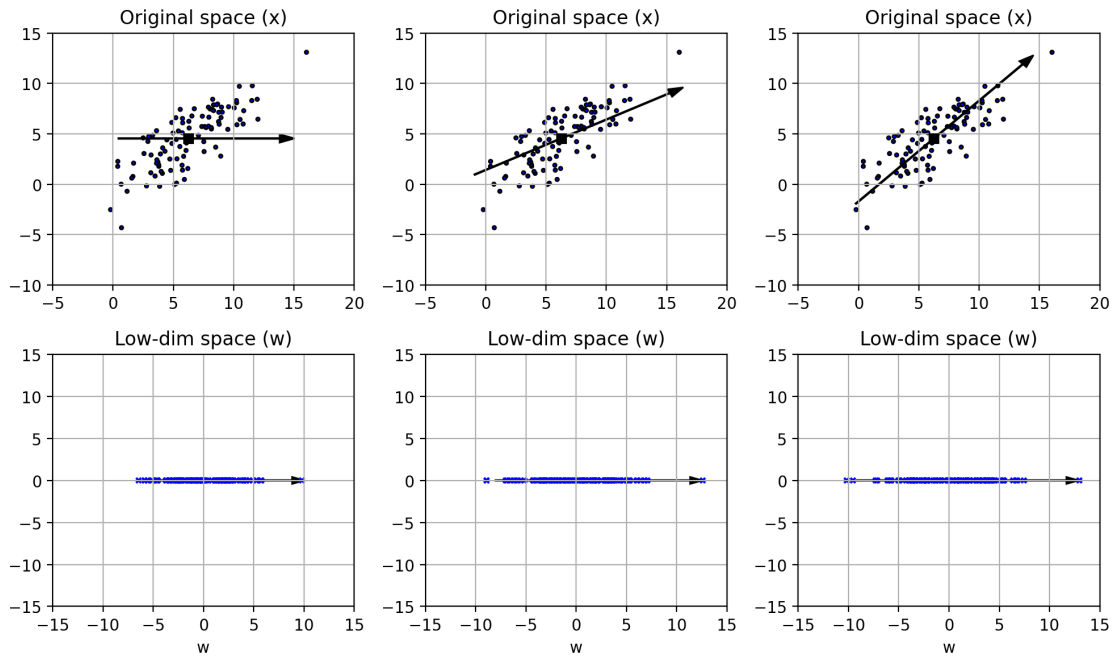
```

plot_basis(Xblob, array([[1, 0.0]]), sp=((2,3,1), (2,3,4)), showleg=False,
↪axbox1=axbox1, axbox2=axbox2)
plot_basis(Xblob, array([[1, 0.5]]), sp=((2,3,2), (2,3,5)), showleg=False,
↪axbox1=axbox1, axbox2=axbox2)
plot_basis(Xblob, array([[1.0, 1.0]]), sp=((2,3,3), (2,3,6)), showleg=False,
↪axbox1=axbox1, axbox2=axbox2)
plt.tight_layout()
plt.close()

```

[5]: vfig

[5]:



- **Goal:** Equivalently, minimize the reconstruction error over all the data points $\{\mathbf{x}_i\}_{i=1}^N$.
 - reconstruction: $\hat{\mathbf{x}}_i = \sum_{j=1}^p w_{i,j} \mathbf{v}_j$

$$\min_{w, \mathbf{v}} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$$

- *constraint:* principal components $\mathbf{v}_j$ are orthogonal (perpendicular) to each other.

$$* \mathbf{v}_j^T \mathbf{v}_i = \begin{cases} 1, i = j \\ 0, i \neq j \end{cases}$$

10 PCA algorithm

- Iteratively select directions that explain the most variance:
 - 1) subtract the mean of the data

- 2) the first PC $\mathbf{v}_1$ is the direction that explains the most variance of the data.
- 3) the second PC $\mathbf{v}_2$ is the direction perpendicular to $\mathbf{v}_1$ that explains the most variance.
- 4) the third PC $\mathbf{v}_3$ is the direction perpendicular to $\{\mathbf{v}_1, \mathbf{v}_2\}$ that explains the most variance.
- 5) ...

11 Solution

- Define the mean-subtracted data as $\bar{\mathbf{x}}_i = \mathbf{x}_i - \mu$.
 - matrix of data $\bar{\mathbf{X}} = [\bar{\mathbf{x}}_1 \cdots \bar{\mathbf{x}}_N]$.
- Consider the covariance matrix of the data: $\Sigma = \frac{1}{N} \sum_i \bar{\mathbf{x}}_i \bar{\mathbf{x}}_i^T = \frac{1}{N} \bar{\mathbf{X}} \bar{\mathbf{X}}^T$
 - the covariance matrix defines ellipses of equal-probability of the Gaussian

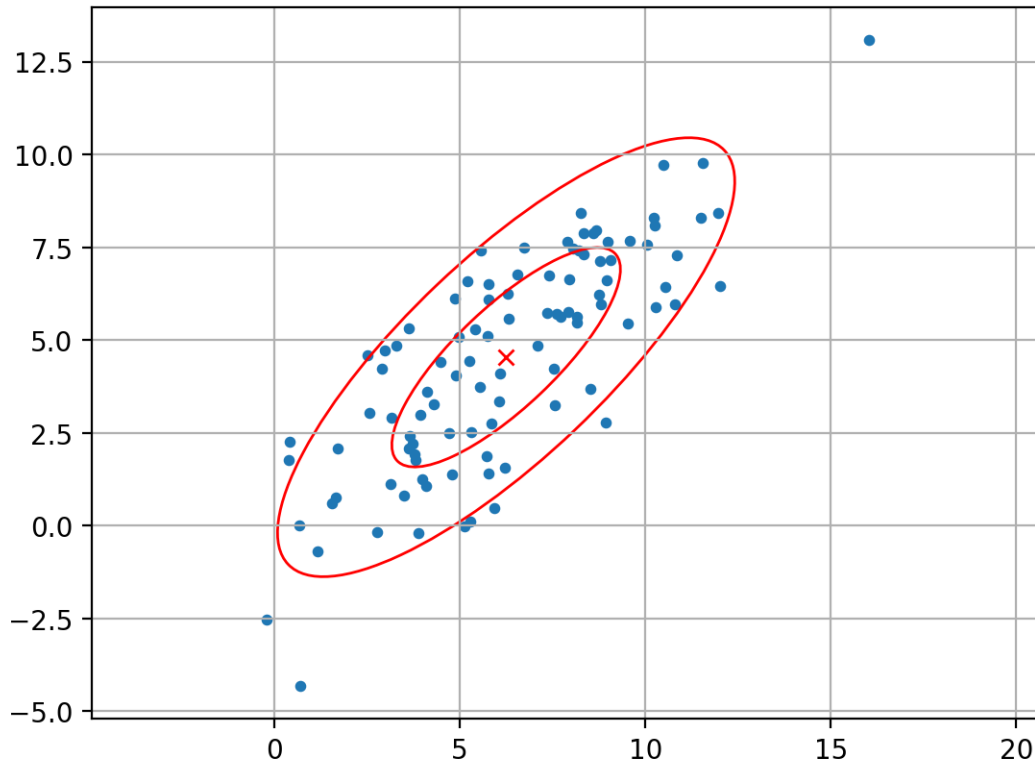
```
[6]: X = Xblob
      Mu = mean(X, axis=0)
      Sigma = cov(X, rowvar=False)

      gfig = plt.figure()
      plt.scatter(X[:,0], X[:,1], s=10)
      plot_ellipse(plt.gca(), (Mu, Sigma), 'r')
      plot_ellipse(plt.gca(), (Mu, 4*Sigma), 'r')
      plt.plot(Mu[0], Mu[1], 'rx', label='mean')
      plt.grid(True)
      plt.axis('equal')
      plt.close()
```

```
[7]: Sigma = cov(X, rowvar=False)
      print(Sigma)
      gfig
```

```
[[9.4987893  7.29218068]
 [7.29218068 8.74288433]]
```

```
[7]:
```

- Consider the eigenvectors and eigenvalues of Σ
 - i-th eigenvector/value pair: $\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$
 - all eigen-pairs: $\Sigma \mathbf{V} = \mathbf{V} \Lambda$
 - * matrix of eigenvectors $\mathbf{V} = [\mathbf{v}_1 \cdots \mathbf{v}_D]$
 - * eigenvectors are orthonormal: $\mathbf{V}^T \mathbf{V} = \mathbf{I}$
 - * diagonal matrix of eigenvalues: $\Lambda = \text{diag}([\lambda_1, \dots, \lambda_D])$
 - **Eigendecomposition** of covariance: $\Sigma = \mathbf{V} \Lambda \mathbf{V}^T$
 - * the eigenvector $\mathbf{v}_i$ is an axis of the ellipse.
 - * the extent of axis $\mathbf{v}_i$ is related to the eigenvalue: $\sqrt{\lambda_i}$

```
[8]: [L,V] = linalg.eig(Sigma)
gfig = plt.figure()
plt.scatter(X[:,0], X[:,1], s=10)
plot_ellipse(plt.gca(), (Mu, Sigma), 'r')
plot_ellipse(plt.gca(), (Mu, 4*Sigma), 'r')
plt.plot(Mu[0], Mu[1], 'rx', label='mean')

E1 = 2*sqrt(L[0])*V[:,0]
E2 = 2*sqrt(L[1])*V[:,1]
plt.arrow(Mu[0], Mu[1], E1[0], E1[1], width=0.2, color='k',
    ↳length_includes_head=True)
plt.text(Mu[0]+E1[0], Mu[1]+E1[1], '$2\sqrt{\lambda_1}\mathbf{v}_1$')
```

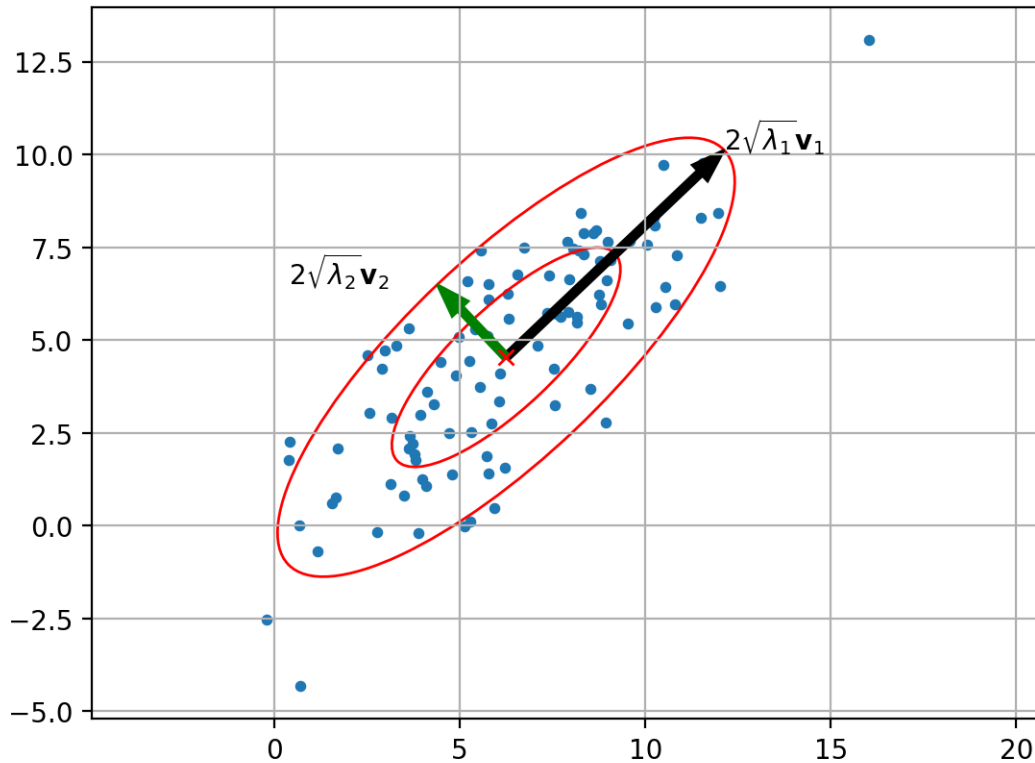
```
plt.arrow(Mu[0], Mu[1], E2[0], E2[1], width=0.2, color='g',
          ↪length_includes_head=True)
plt.text(Mu[0]+E2[0]-4, Mu[1]+E2[1], '$2\sqrt{\lambda_2}\mathbf{v}_2$', )
plt.grid(True)
plt.axis('equal')
plt.close()
#plt.axis(axboxb)
#plt.close()
```

```
<>:11: SyntaxWarning: invalid escape sequence '\s'
<>:13: SyntaxWarning: invalid escape sequence '\s'
<>:11: SyntaxWarning: invalid escape sequence '\s'
<>:13: SyntaxWarning: invalid escape sequence '\s'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2209011898.py:1
1: SyntaxWarning: invalid escape sequence '\s'
    plt.text(Mu[0]+E1[0], Mu[1]+E1[1], '$2\sqrt{\lambda_1}\mathbf{v}_1$')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2209011898.py:1
3: SyntaxWarning: invalid escape sequence '\s'
    plt.text(Mu[0]+E2[0]-4, Mu[1]+E2[1], '$2\sqrt{\lambda_2}\mathbf{v}_2$', )
```

```
[9]: [L,V] = linalg.eig(Sigma)
      print("lambda = ", L)
      print("v1 = ", V[:,0])
      print("v2 = ", V[:,1])
      gfig
```

```
lambda = [16.42280553  1.8188681 ]
v1 = [0.72517596 0.68856359]
v2 = [-0.68856359 0.72517596]
```

[9]:



- Thus the solution to PCA is to select the eigenvectors with largest eigenvalues first.
- To reduce to K dimensions
 - 1) subtract the mean of the data
 - 2) compute the covariance matrix Σ of the data.
 - 3) sort the eigenvector/values of Σ by $\lambda_1 > \lambda_2 > \dots > \lambda_d$
 - 4) select the top K eigenvectors: $\mathbf{V} = [\mathbf{v}_1 \dots \mathbf{v}_K]$
 - 5) project the data onto the PCA basis: $\mathbf{w}_i = \mathbf{V}^T \mathbf{x}_i \in \mathbb{R}^K$
- Reconstruction: $\hat{\mathbf{x}}_i = \mathbf{V} \mathbf{w}_i$

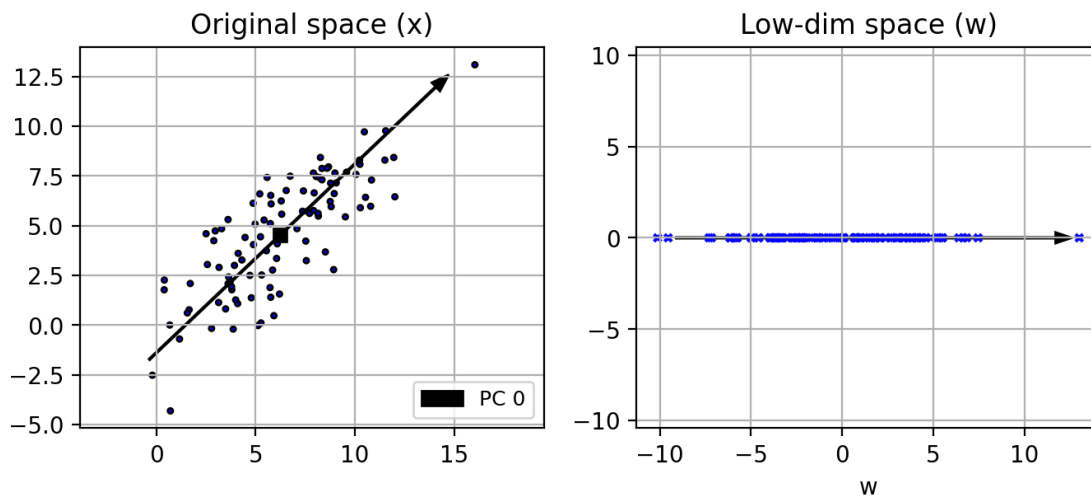
12 Example - 1 PC

```
[10]: X = Xblob

# run PCA
pca = decomposition.PCA(n_components=1)
W = pca.fit_transform(X) # returns the coefficients

v = pca.components_ # the principal component vector
m = pca.mean_       # the data mean
```

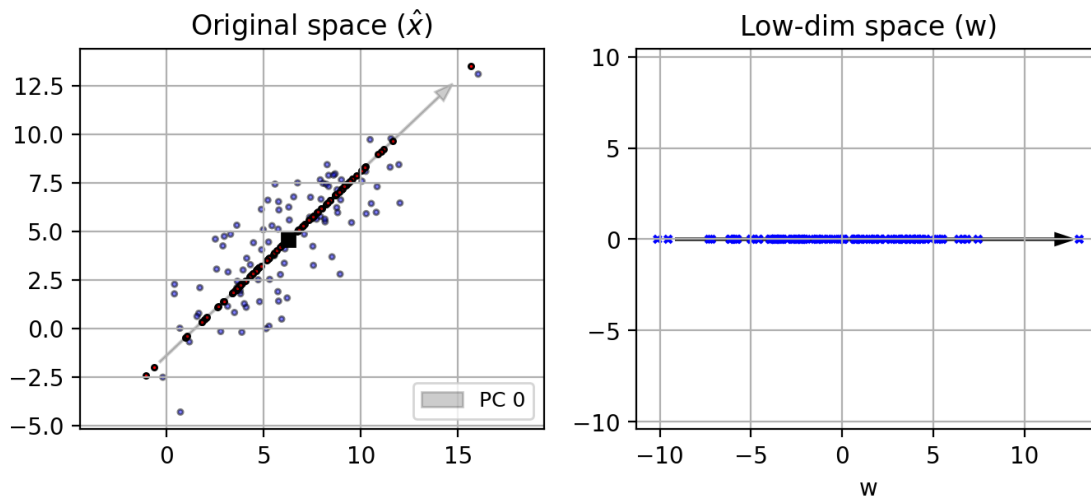
```
plt.figure(figsize=(8,3))
plot_basis(X, v);
```



13 Example - 1 PC reconstruction

- reconstructed $\hat{x}_i$ are projected onto the PC

```
[11]: plt.figure(figsize=(8,3))
plot_basis(X, v, recon=True);
```

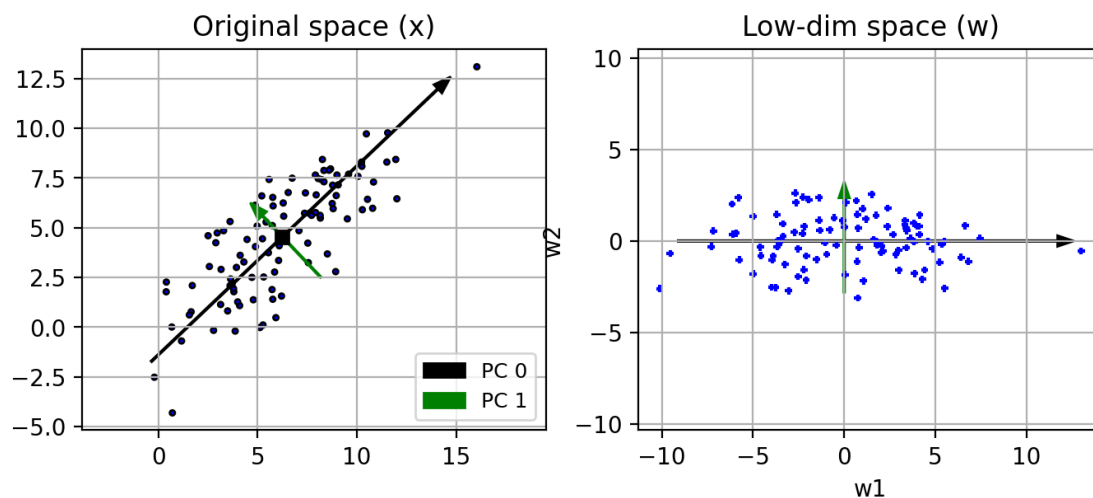


14 Example - 2 PC

```
[12]: # run PCA
pca = decomposition.PCA(n_components=2)
W = pca.fit_transform(X)

v = pca.components_ # the principal component vector
m = pca.mean_       # the data mean

plt.figure(figsize=(8,3))
plot_basis(X, v);
```



15 Example on Iris data

- 2D (petal length, sepal width) to 1D

```
[13]: # example on Iris
axbox = [0, 8, 1.5, 5]

rbow = plt.get_cmap('rainbow')

# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris3.csv', delimiter=',', skiprows=1)

Xiris = irisdata[:,0:2] # the first two columns are features (petal length,
    ↪sepal width)
Yiris = irisdata[:,2]   # the third column is the class label (setosa=0,
    ↪versicolor=1, virginica=2)
```

```

X = Xiris
Y = Yiris

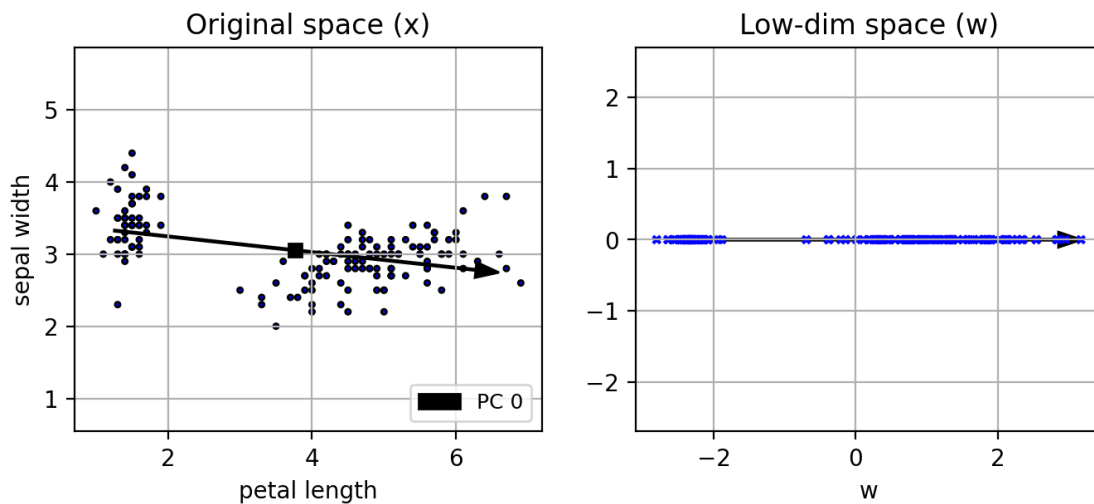
pca = decomposition.PCA(n_components=1)
W = pca.fit_transform(X)

ifig = plt.figure(figsize=(8,3))
axp = plot_basis(X, pca.components_)
plt.sca(axp[0])
plt.xlabel('petal length')
plt.ylabel('sepal width')
plt.close()

```

[14]: ifig

[14]:



- 4D to 2D
- mostly preserves the structure of the classes.

```

[15]: # get data
iris = datasets.load_iris()
X = iris.data
Y = iris.target

# run PCA
pca = decomposition.PCA(n_components=2)
W = pca.fit_transform(X)
print(iris.feature_names)
print(pca.components_)

```

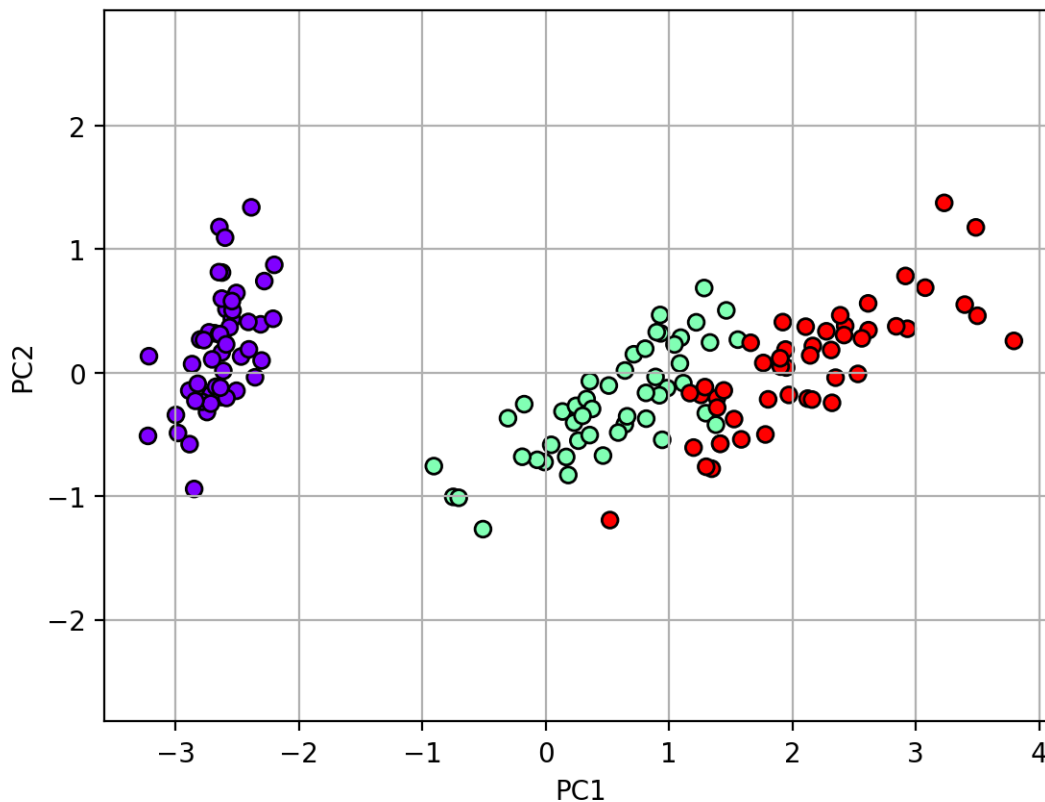
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width

```
(cm)']
[[ 0.36138659 -0.08452251  0.85667061  0.3582892 ]
 [ 0.65658877  0.73016143 -0.17337266 -0.07548102]]
```

```
[16]: i4fig = plt.figure()
plt.scatter(W[:,0], W[:,1], c=Y, cmap=rbow, edgecolors='k')
plt.axis('equal'); plt.grid(True)
plt.xlabel('PC1'); plt.ylabel('PC2');
plt.close()
```

```
[17]: i4fig
```

```
[17]:
```



16 How to choose the number of principal components?

- Two methods to set the number of components p :
 - preserve some percentage of the variance (e.g., 95%).
 - whatever works well for our final task (e.g., classification, regression).

17 Handwritten digits data

- 1797 images of handwritten digits 0-9

- each image is 8x8
- flattened into a 64 dimensional vector

```
[18]: # get digit data
digits = datasets.load_digits()
Xdigits = float64(digits.data)
Ydigits = digits.target

print(Xdigits.shape)
```

(1797, 64)

```
[19]: def show_array_as_imgs(X, imgsize, plotwidth, titles=None, Xmask=None,
    ↪ vmms=None, cmaps=None):
    "show each row of array X as an image"

    if X.ndim == 1:
        numX = 1
    else:
        numX = X.shape[0]

    height = int(ceil(numX/plotwidth))
    for n in range(numX):
        if (Xmask is None) or (Xmask[n]):
            plt.subplot(height,plotwidth,n+1)
            if numX > 1:
                I = X[n].reshape(imgsize)
            else:
                I = X.reshape(imgsize)

            D = {}
            if vmms is not None:
                D['vmin'] = vmms[n][0]
                D['vmax'] = vmms[n][1]
            if cmaps is not None:
                D['cmap'] = cmaps[n]
            plt.imshow(I, interpolation='nearest', **D)
            if cmaps is None:
                plt.gray()
            if titles is not None:
                plt.title(str(titles[n]))
            plt.gca().axis.set_ticklabels([])
            plt.gca().yaxis.set_ticklabels([])

    imgsize = (8,8)
    # show examples
    dfig = plt.figure(figsize=(8,3.5))
    myi = range(0,1797,190)
```



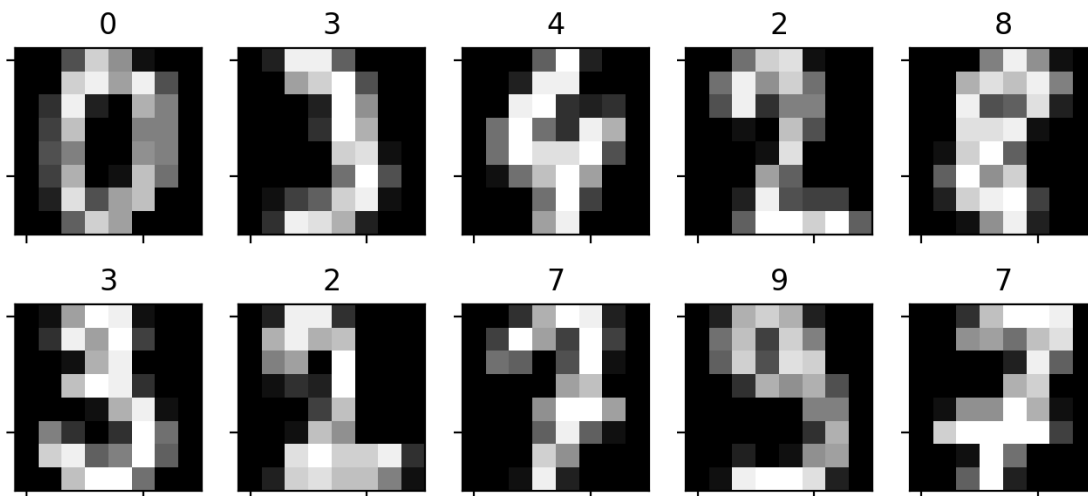
```

vmin = amin(Xdigits)
vmax = amax(Xdigits)
show_array_as_imgs(Xdigits[myi], imgsize, 5, Ydigits[myi],
    ↪vmms=[(vmin,vmax)]*len(myi))
plt.close()

```

[20]: dfig

[20]:



18 Run PCA on the data

- split data into training and testing sets.
- run PCA on training set, apply to test set

```

[21]: # randomly split data into 80% train and 20% test set
trainX, testX, trainY, testY = \
    model_selection.train_test_split(Xdigits, Ydigits,
        train_size=0.5, test_size=0.5, random_state=4487)
Xdim = Xdigits.shape[1]

# run PCA
pca = decomposition.PCA() # default: n_components=dimension
W = pca.fit_transform(trainX) # fit the training set
Wt = pca.transform(testX) # use the pca model to transform the test set

```

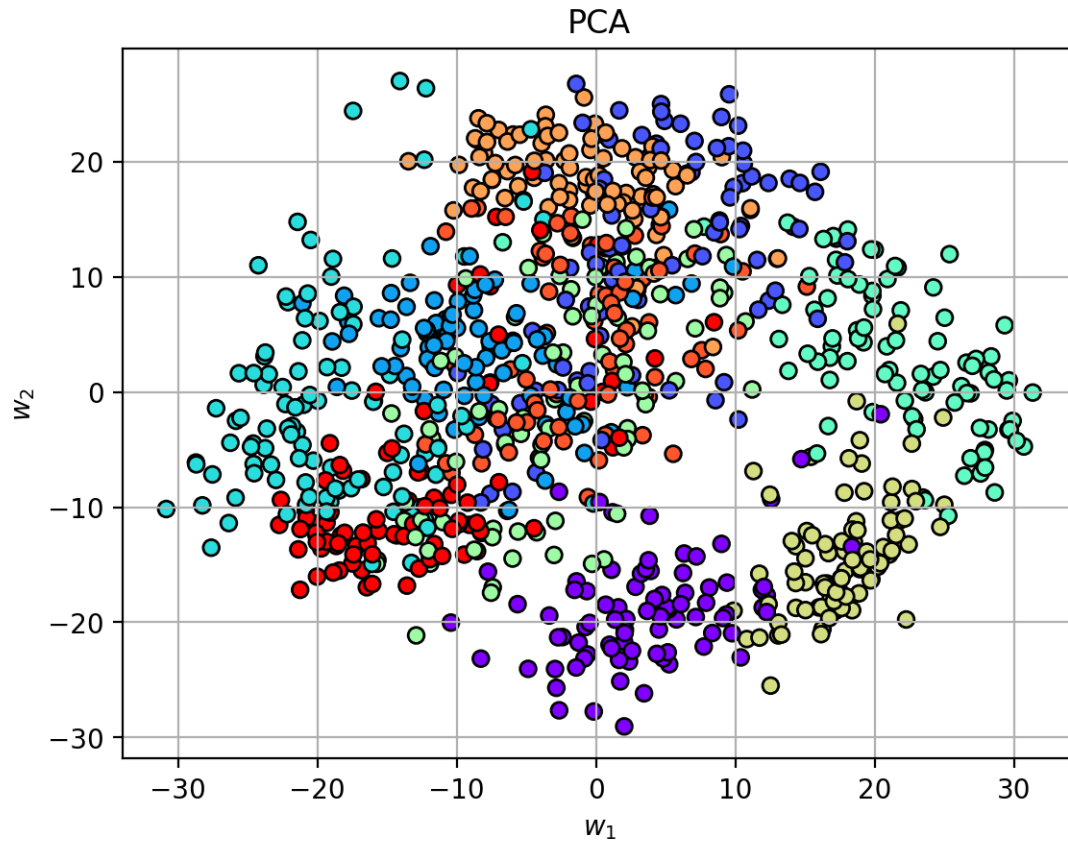
- Visualize the coefficients for the first two PCs.
 - grouping of different digits is sometimes preserved

```

[22]: plt.figure()
plt.scatter(W[:,0], W[:,1], c=trainY, cmap=rbow, edgecolors='k')
plt.xlabel('$w_1$'); plt.ylabel('$w_2$')

```

```
plt.title('PCA'); plt.grid(True);
```



- Look at the mean and principal components

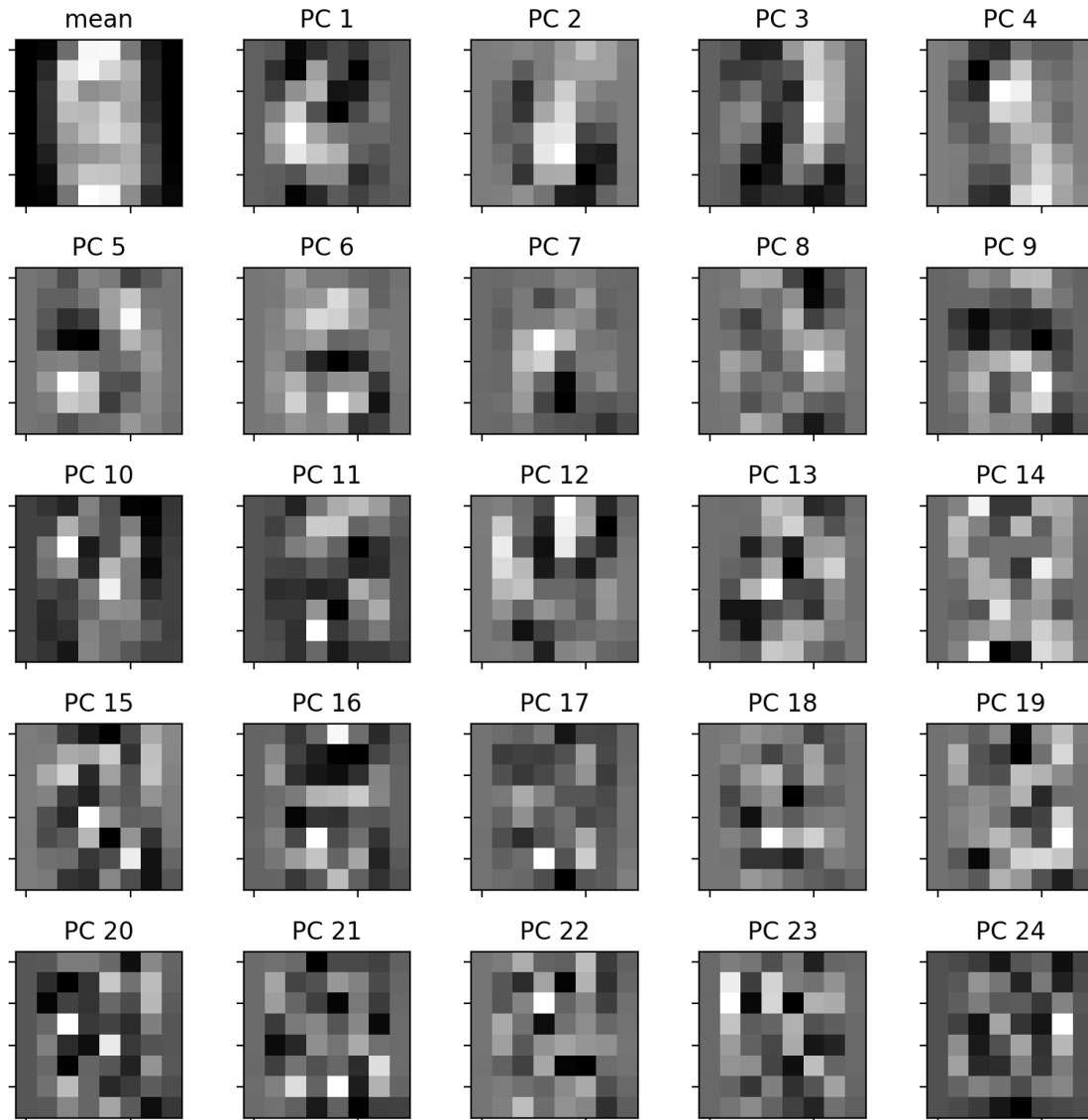
```
[23]: # plot the mean and the PCs
pcfig = plt.figure(figsize=(8,8))

tmpX = vstack((pca.mean_, pca.components_[range(24)]))
tmpT = ['mean']
for j in range(24):
    tmpT.append("PC " + str(j+1))

show_array_as_imgs(tmpX, imgsize, 5, tmpT)
plt.tight_layout()
plt.close()
```

```
[24]: pcfig
```

```
[24]:
```



- Reconstruction of a digit image from PC coefficients
 - green/red corresponds to positive/negative values
 - using more PCs will make the reconstruction better

```
[25]: myX = trainX[0]
myW = W[0]
nump = 6

myR = zeros( (3*(nump+1), 64) )
myR[0] = myX
myT = ['Original', None, None]
Xmask = [True, False, False]
```

```

vmms = [(vmin,vmax), None, None]

bwrcmap = matplotlib.colors.LinearSegmentedColormap.from_list('gkr',
    colors=['r', 'k', 'g'])
gcmmap = plt.get_cmap('gray')
mycmaps = [gcmmap, None, None]

for i in range(nump):
    basei = (i+1)*3
    if i==0:
        myR[basei] = pca.mean_
        myT.append('mean')
        vmms.append((vmin,vmax))
        mycmaps.append(gcmmap)
    else:
        # take from previous result
        myR[basei] = myR[basei-1]
        myT.append(myT[basei-1][2:])
        vmms.append((vmin,vmax))
        mycmaps.append(gcmmap)

    # add PC
    myR[basei+1] = myW[i] * pca.components_[i]
    myT.append('+ {:.3f}*PC{}'.format(myW[i], i+1))
    vmms.append((-vmax/2,vmax/2))
    mycmaps.append(bwrcmap)

    # new recon
    myR[basei+2] = myR[basei+1] + myR[basei+0]
    myT.append('= Recon' + str(i+1))
    vmms.append((vmin,vmax))
    mycmaps.append(gcmmap)

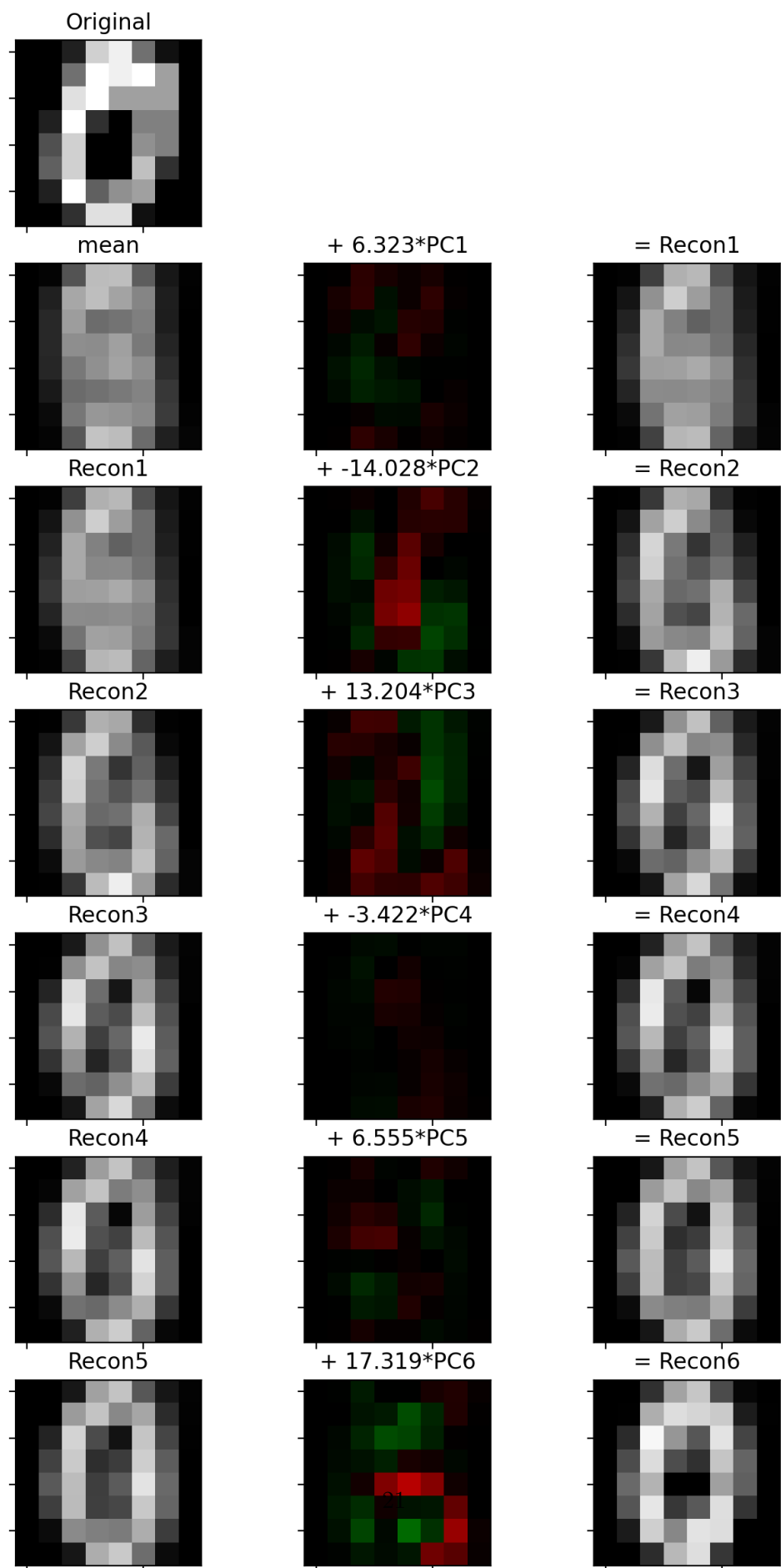
    Xmask.append(True); Xmask.append(True); Xmask.append(True)

reconfig = plt.figure(figsize=(8,15))
show_array_as_imgs(myR, imgsize, 3, myT, Xmask=Xmask, vmms=vmms, cmaps=mycmaps)
plt.close()

```

[26]: reconfig

[26]:

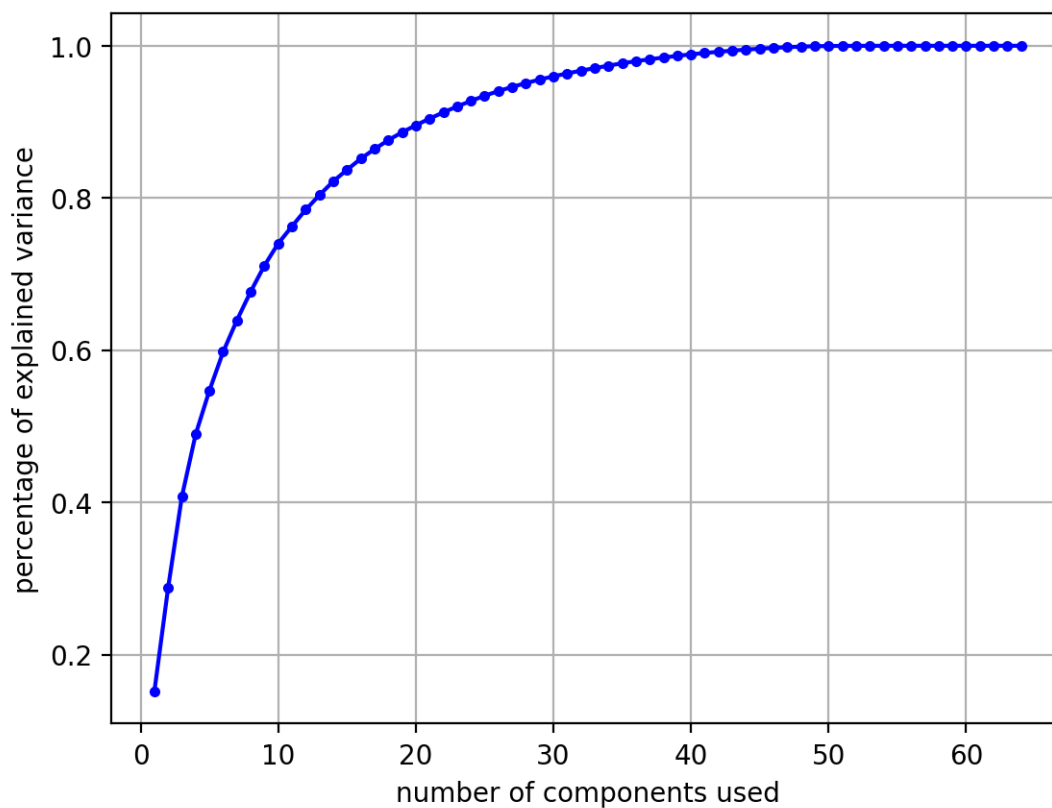


19 Explained variance

- each PC explains a percentage of the original data
 - this is called the *explained variance*.
 - PCs are already sorted by explained variance from highest to lowest
- pick the number of PCs to get a certain percentage of explained variance
 - typically 95%

```
[27]: ev      = pca.explained_variance_ratio_  # variance explained by each component
      cumev = cumsum(ev)                    # cumulative explained variance

      plt.plot(range(1,Xdim+1), cumev, 'b.-')
      plt.grid(True)
      plt.xlabel('number of components used')
      plt.ylabel('percentage of explained variance');
```



20 Task-dependent Selection

- use results on the final task (in this case classification) to select the best number of components
- Note: we don't need to rerun PCA for each number of components
 - just select the subset of PCs based on the number of components desired.

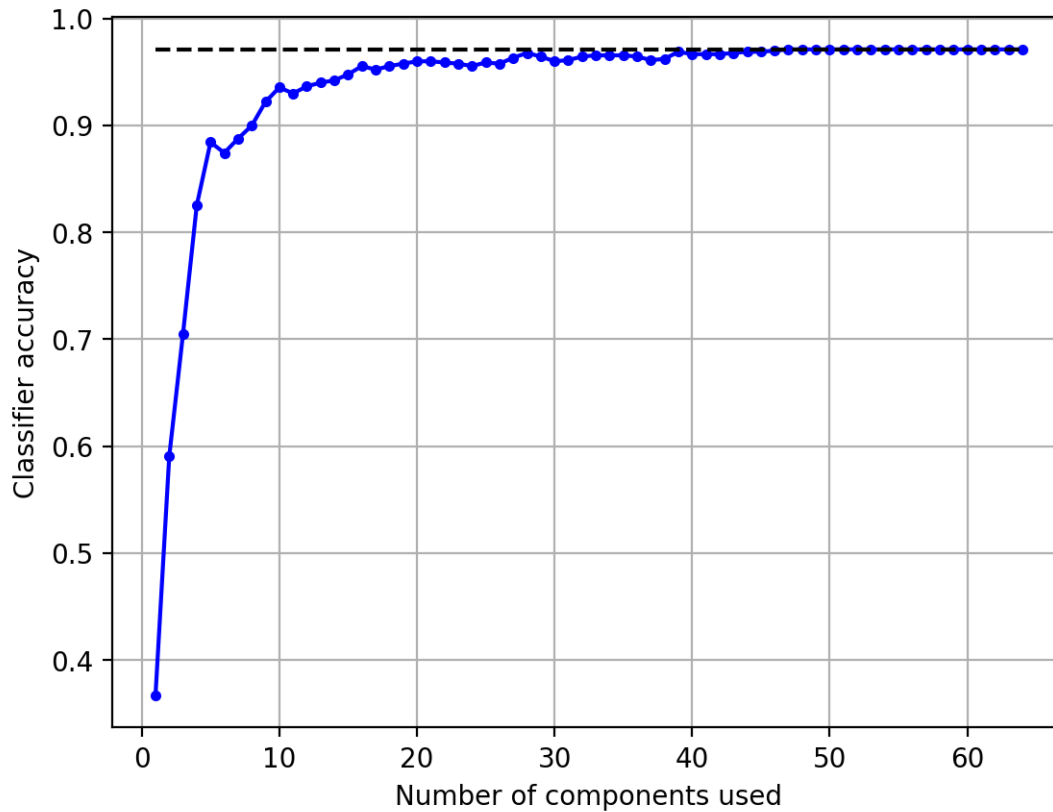
```
[28]: acc = zeros(Xdim)
      for j in range(Xdim):
          # extract the subset of PC weights [0,j]
          Wnew      = W[:,0:(j+1)]
          Wnewtest  = Wt[:,0:(j+1)]

          # train classifier
          clf = svm.SVC(kernel='linear', C=1)
          clf.fit(Wnew, trainY)

          # test classifier
          Ypred = clf.predict(Wnewtest)
          acc[j] = metrics.accuracy_score(testY, Ypred)
```

- classification accuracy is stable after using 20 PCs.
 - not much loss in performance if using only 20 PCs.

```
[29]: # make a plot
      plt.plot(range(1,Xdim+1), acc, '.b-')
      plt.plot([1,Xdim], [acc.max(), acc.max()], 'k--')
      plt.grid(True)
      plt.xlabel('Number of components used')
      plt.ylabel('Classifier accuracy');
```



21 Denoising

- the low-dim PCA space summarizes the important variations of the data.
- the original image can be denoised by:
 - 1) project into the low-dimensional space to get PCA coefficients
* (keep only important variations)
 - 2) reconstruct an image from the PCA coefficients

```
[30]: # add noise to data
noisyX = trainX + 2*random.normal(size=trainX.shape)
noisyXt = testX + 2*random.normal(size=testX.shape)

# learn PCA
pca = decomposition.PCA(n_components=10)
pca.fit(noisyX) # fit the training set

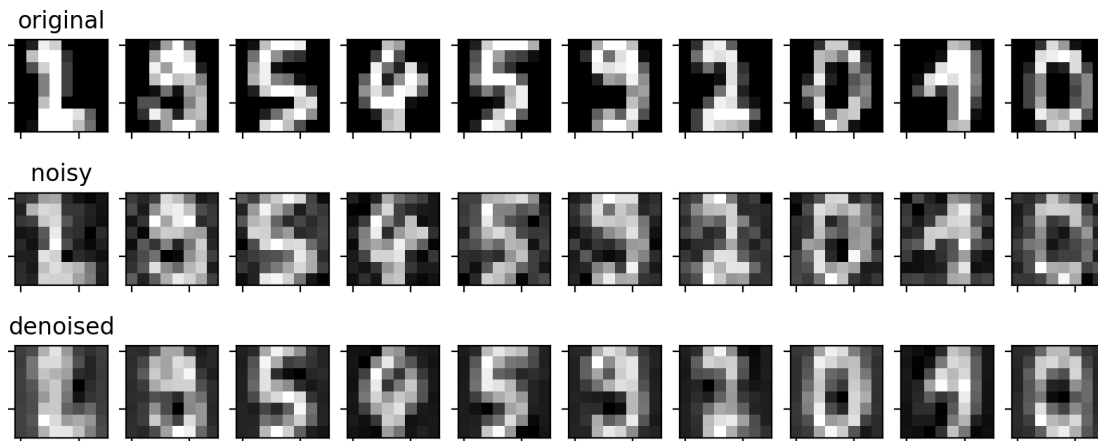
# transform and reconstuct
testW = pca.transform(noisyXt)
testXr = pca.inverse_transform(testW) # reconstruction
```



```
[31]: myi = [0, 300, 500, 800, 801, 100, 200, 301, 401, 501]
      tmpT = [""]*30
      tmpT[0] = "original"
      tmpT[10] = 'noisy'
      tmpT[20] = "denoised"
      rfig = plt.figure(figsize=(10,4))
      show_array_as_imgs(array([*testX[myi], *noisyXt[myi], *testXr[myi]]), imgsize,
      ↪10, tmpT)
      plt.close()
```

```
[32]: rfig
```

```
[32]:
```



22 Random Projections

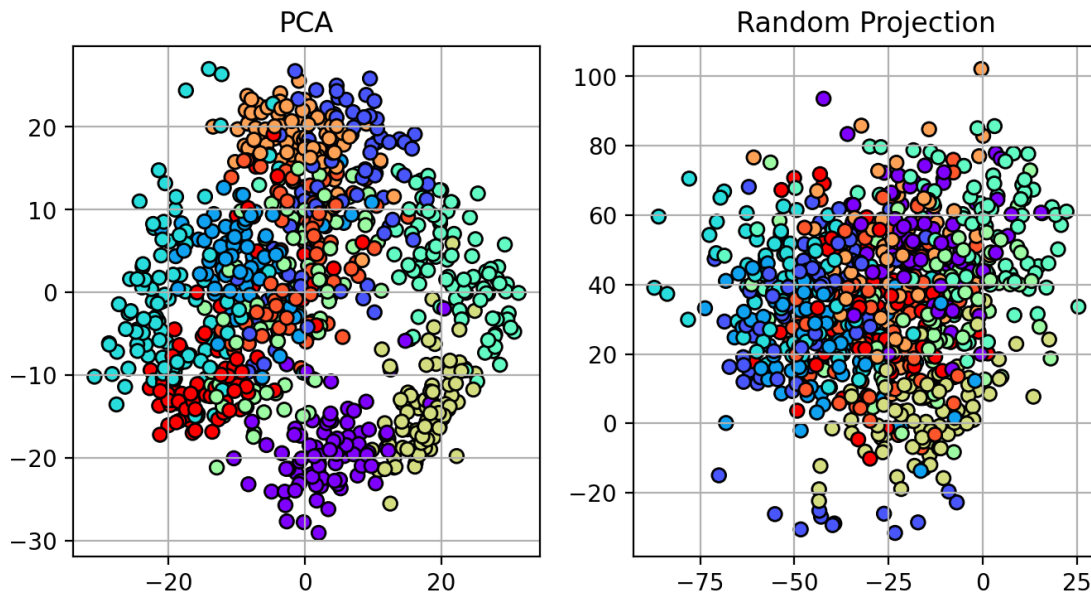
- If the data is very high-dimensional, then it might take too many calculations to do PCA.
 - Complexity: $O(dk^2)$, d is the dimension, k is the number of components
- Do we really need to estimate the principal components to reduce the dimension?
- **Solution:**
 - We can generate random basis vectors and use those.
 - * Each entry of $\mathbf{v}_j$ sampled from a Gaussian.
 - This will save a lot of time.
 - Random Projections can reduce computation at the expense of losing some accuracy in the points (adding noise).

```
[33]: # project the digits data with Random Projection
      rp = random_projection.GaussianRandomProjection(n_components=2,
      ↪random_state=4487)
      Wrp = rp.fit_transform(trainX)
```

```
[34]: rfig = plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.scatter(W[:,0], W[:,1], c=trainY, cmap=rbow, edgecolors='k'); plt.grid(True)
plt.title('PCA')
plt.subplot(1,2,2)
plt.scatter(Wrp[:,0], Wrp[:,1], c=trainY, cmap=rbow, edgecolors='k'); plt.
    ↪grid(True)
plt.title('Random Projection');
plt.close()
```

```
[35]: rfig
```

```
[35]:
```



- Okay, but is it good?
 - One way to measure “goodness” is to see if the structure of the data is preserved.
 - In other words, are distances between points preserved in the transformed data?
- **Answer:**
 - Yes!
 - According to the *Johnson-Lindenstrauss lemma*, carefully selecting the distribution of the random projection matrices will preserve the pairwise distances between any two samples of the dataset, within some error *epsilon*: $(1-\epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 < \|\mathbf{w}_i - \mathbf{w}_j\|^2 < (1+\epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2$

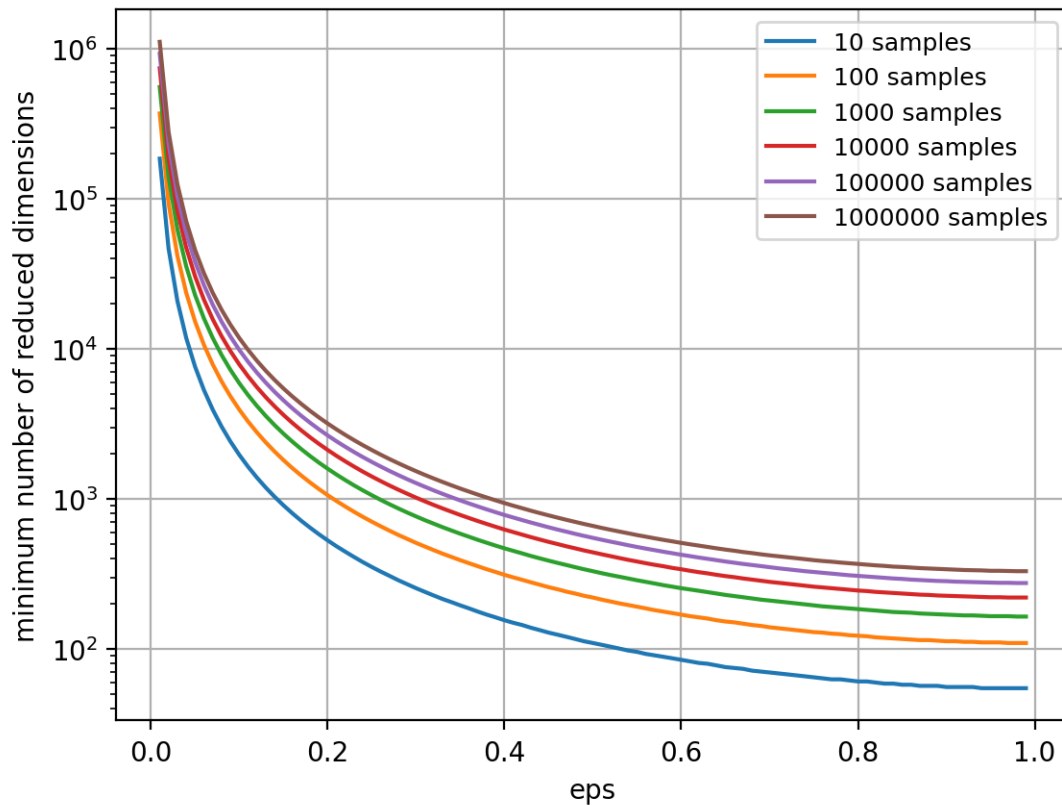
```
[36]: jlfig = plt.figure()
ns = 10**arange(1,7)
epss = arange(0.01, 1.00, 0.01)
for n in ns:
    mn = random_projection.johnson_lindenstrauss_min_dim(n_samples=n, eps=epss)
    plt.semilogy(epss, mn, label=str(n)+" samples")
```

```
plt.grid(True)
plt.xlabel('eps')
plt.ylabel('minimum number of reduced dimensions')
plt.legend(fontsize=9)
plt.close()
```

- the minimum reduced dimension p to guarantee an ϵ error depends on the number of samples.
 - (actually, this is fairly conservative)

[37]: jlfig

[37]:



- Example

```
[38]: # generate random data
# (dimension=10000, samples=100)
X = random.rand(100,10000)

# fit to 500 components
rp = random_projection.GaussianRandomProjection(n_components=500,
↪random_state=4487)
Wrp = rp.fit_transform(X)
```

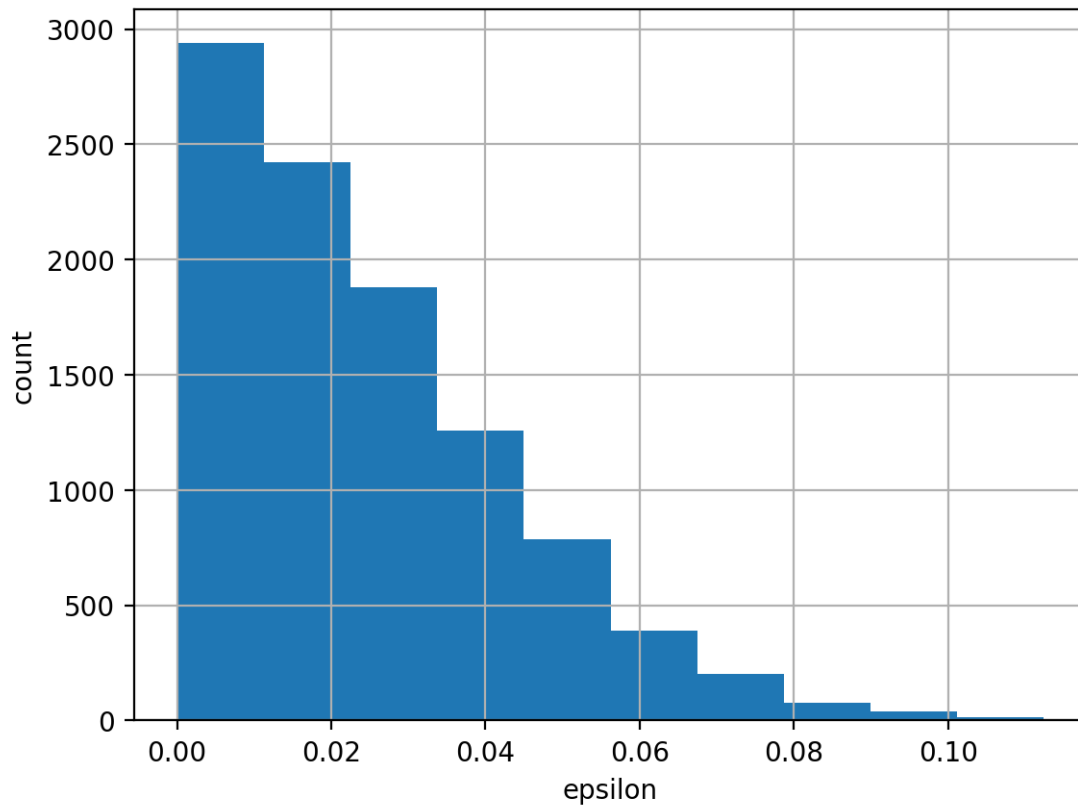
```
[39]: # Calculate the pairwise distances
D = metrics.pairwise.euclidean_distances(X, X)
Drp = metrics.pairwise.euclidean_distances(Wrp, Wrp)

# calculate epsilon
epss = abs(Drp/D-1)
epss[isnan(epss)] = 0 # remove 0/0
efig = plt.figure()
plt.hist(epss.flatten())
plt.xlabel('epsilon')
plt.ylabel('count')
plt.grid(True)
plt.close()
```

```
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/3595013772.py:6
: RuntimeWarning: invalid value encountered in divide
  epss = abs(Drp/D-1)
```

```
[40]: efig
```

```
[40]:
```



23 Sparse Random Projection

- More computation can be saved by using a *sparse* random projection matrix
 - “sparse” means that many entries in the basis vector are zero, so we can ignore those entries when multiplying.

```
[41]: # project the digits data with Random Projection
srp = random_projection.SparseRandomProjection(n_components=500,
↳ random_state=4487)
Wsrp = srp.fit_transform(X)
```

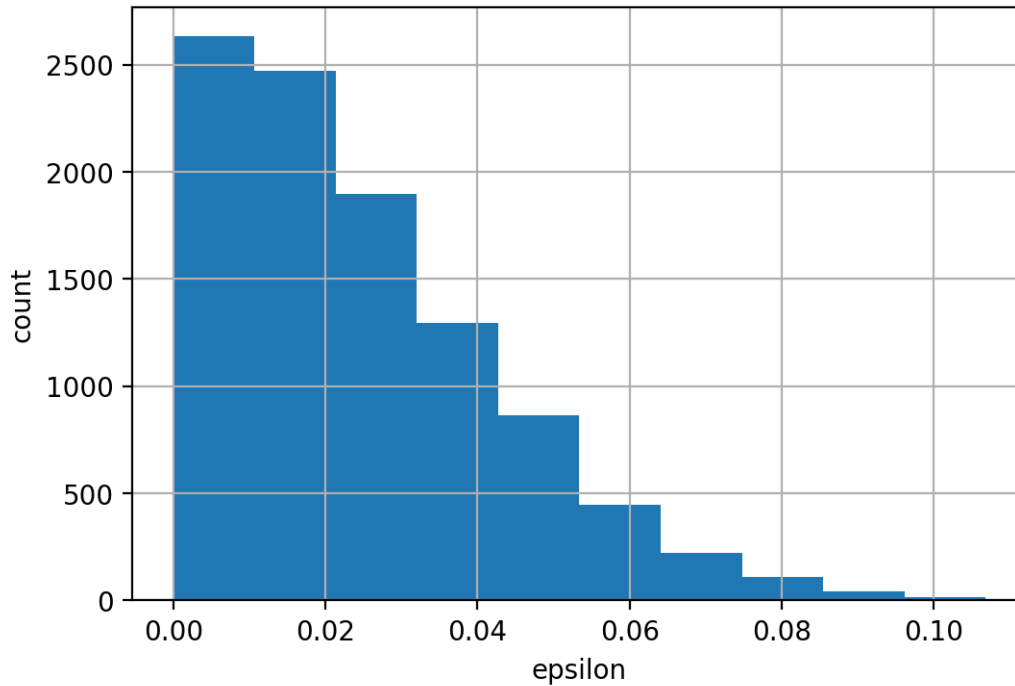
```
[42]: # calculate pairwise distances
D = metrics.pairwise.euclidean_distances(X, X)
Dsrp = metrics.pairwise.euclidean_distances(Wsrp, Wsrp)

# calculate epsilon
epss = abs(Dsrp/D-1)
epss[isnan(epss)] = 0 # remove 0/0
efig = plt.figure(figsize=(6,4))
plt.hist(epss.flatten())
plt.xlabel('epsilon')
plt.ylabel('count')
plt.grid(True)
plt.close()
```

```
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_45815/2904337220.py:6
: RuntimeWarning: invalid value encountered in divide
  epss = abs(Dsrp/D-1)
```

```
[43]: efig
```

```
[43]:
```



24 Question

- Suppose we have the below classification problem...
- We want to reduce the data to 1 dimension using PCA.
 - What is the first PC?

```
[44]: # example of PCA problem
random.seed(38)
Xbars,Ybars = datasets.make_blobs(300,2,centers=[[-5,0], [5,0]], cluster_std=0.
↪8)
Xbars[:,0] /=8

X = Xbars
Y = Ybars

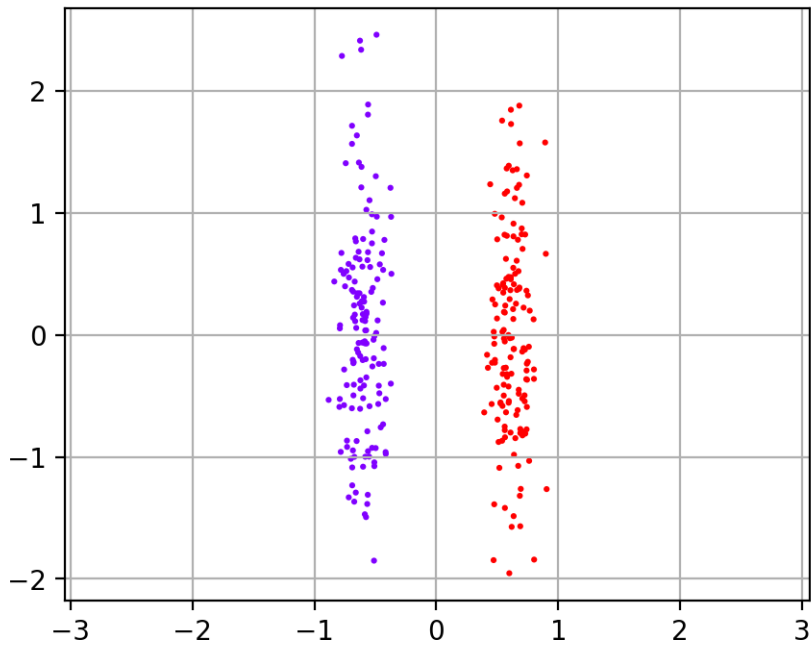
efig = plt.figure(figsize=(5,4))
plt.scatter(X[:,0], X[:,1], c=Y, cmap=rbow, s=5, edgecolor='none')
plt.axis('equal'); plt.grid(True)
plt.close()

e2fig = plt.figure(figsize=(8,4))
pca = decomposition.PCA(n_components=1)
W = pca.fit_transform(X)
plot_basis(X, pca.components_, Y=Y)
```

```
plt.close()
```

```
[45]: efig
```

```
[45]:
```

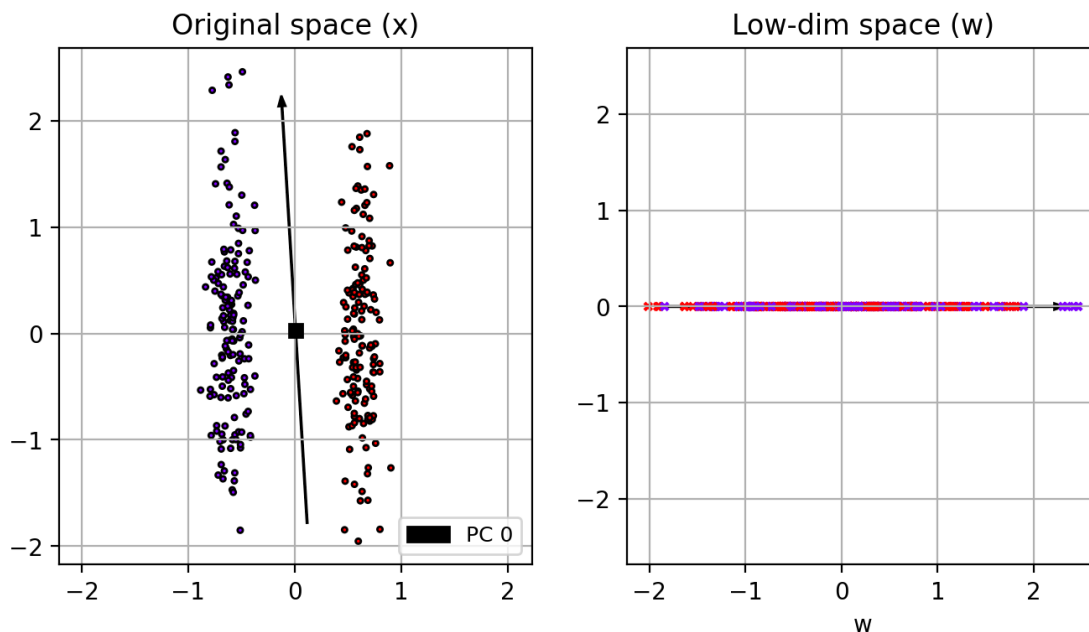


25 Answer

- first PC is along the direction of most variance.
 - collapses the two classes together!

```
[46]: e2fig
```

```
[46]:
```



26 Problem with Unsupervised Methods

- If our end goal is classification, preserving the variance sometimes won't help!
 - PCA doesn't consider which class the data belongs to.
 - When the “classification” signal is less than the “noise”, PCA will make classification more difficult.

27 Fisher's Linear Discriminant (FLD)

- Supervised dimensionality reduction
- Also called “*Linear Discriminant Analysis*” (LDA)
- **Goal:** find a lower-dim space so as to minimize the class overlap (or maximize the class separation).
 - data from each class is modeled as a Gaussian.
 - requires the class labels

28 Problem setup

- Input space: class means μ_j and covariance (scatter) matrices $\mathbf{S}_j$.
- Projected space: class means $m_j = \mathbf{w}^T \mu_j$, and scatter $s_j = \mathbf{w}^T \mathbf{S}_j \mathbf{w}$

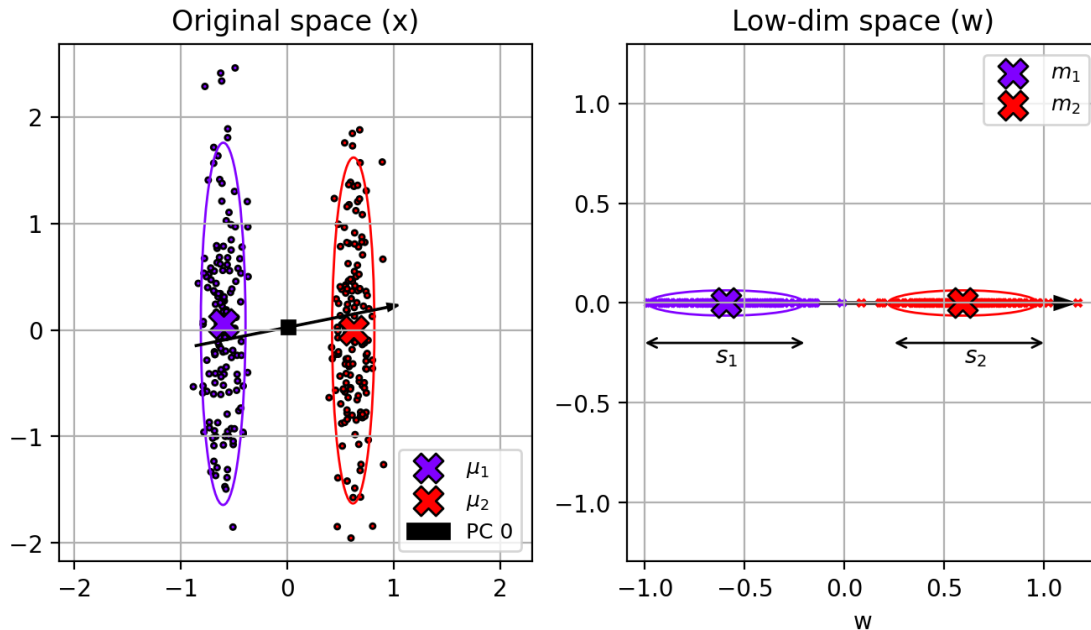
```
[47]: lfig = plt.figure(figsize=(8,4))
      plot_basis(X, array([[1.,.2]]), Y=Y, showellipse=True)
      #plt.arrow(-1,-0.2,0.8,0, width=0.03, c=)
```



```
plt.annotate(text='', xy=(-1,-0.2), xytext=(-0.2,-0.2),  
             ↪arrowprops=dict(arrowstyle='<->', shrinkA=0, shrinkB=0))  
plt.annotate(text='', xy=(0.25,-0.2), xytext=(1,-0.2),  
             ↪arrowprops=dict(arrowstyle='<->', shrinkA=0, shrinkB=0))  
plt.text(-0.65,-0.3,"$s_1$")  
plt.text(0.6,-0.3,"$s_2$")  
plt.close()
```

[48]: lfig

[48]:

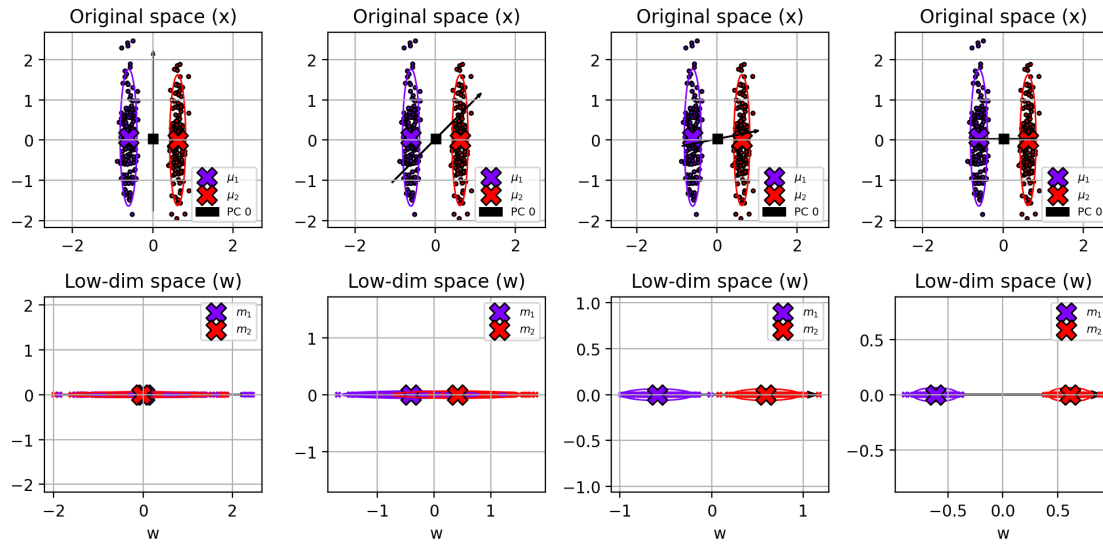


- **Idea:** make the projected points in each class as compact as possible
 - maximize the distance between the projected means
 - minimize the projected variances

```
[49]: efig = plt.figure(figsize=(10,5))  
plot_basis(X, array([[0.,1.]]), Y=Y, showellipse=True, sp=((2,4,1), (2,4,5)),  
           ↪legfont=7)  
plot_basis(X, array([[1.,1.]]), Y=Y, showellipse=True, sp=((2,4,2), (2,4,6)),  
           ↪legfont=7)  
plot_basis(X, array([[1.,.2]]), Y=Y, showellipse=True, sp=((2,4,3), (2,4,7)),  
           ↪legfont=7)  
plot_basis(X, array([[1.,0.]]), Y=Y, showellipse=True, sp=((2,4,4), (2,4,8)),  
           ↪legfont=7)  
plt.tight_layout()  
plt.close()
```

[50]: efig

[50]:



- Fisher's linear discriminant (FLD)
 - Problem:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmax}} \frac{(m_1 - m_2)^2}{s_1 + s_2}$$

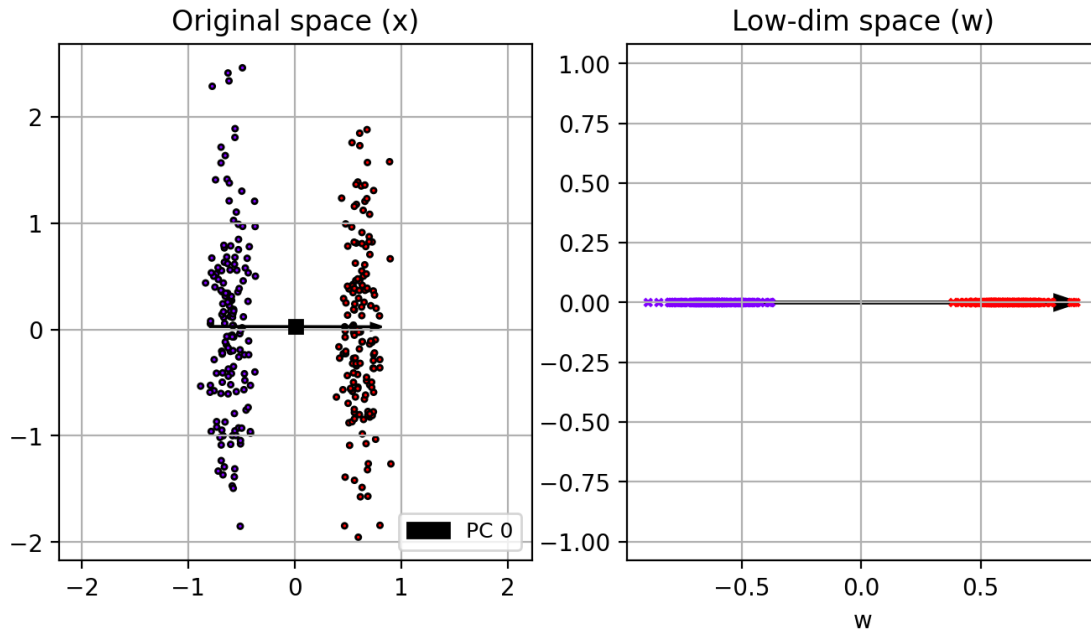
- Solution:

$$\mathbf{w}^* = (\mathbf{S}_1 + \mathbf{S}_2)^{-1}(\mu_1 - \mu_2)$$

```
[51]: # example of FLD projection (using LDA name)
fld = discriminant_analysis.LinearDiscriminantAnalysis(n_components=1)
W = fld.fit_transform(X, Y)

v = fld.coef_ # the basis vectors

plt.figure(figsize=(8,4))
plot_basis(X, v, Y=Y);
```



29 On Iris data

- 4D vector to 2D vector
- FLD forms more compact classes
- With FLD, classes have less overlap if only using 1st basis vector.

```
[52]: # get data
X = iris.data # 4D data
Y = iris.target

# run PCA
pca = decomposition.PCA(n_components=2)
Wpca = pca.fit_transform(X)

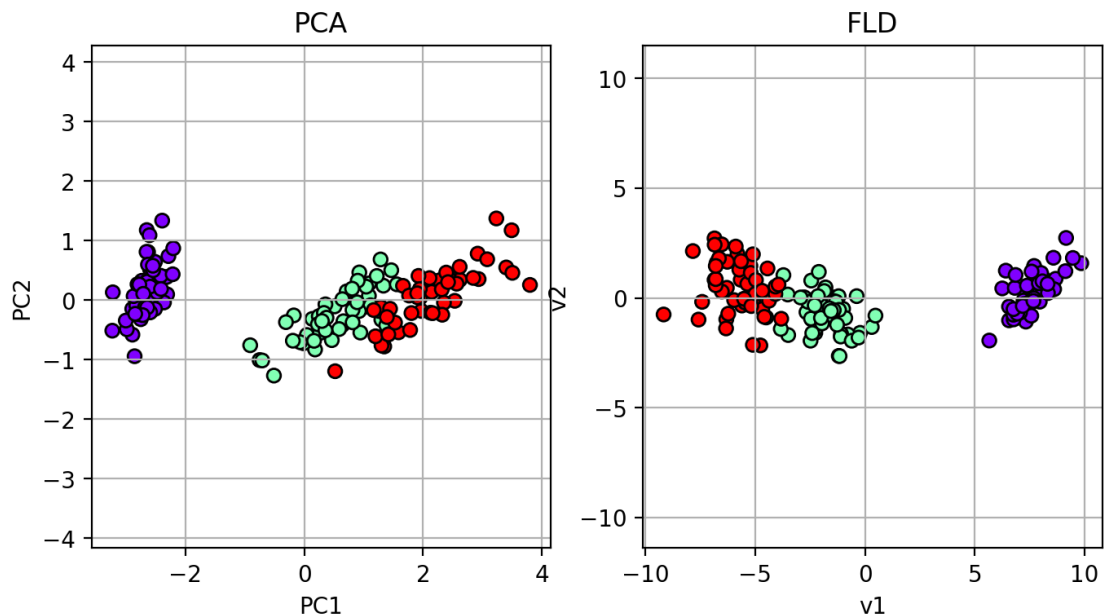
# run LDA
fld = discriminant_analysis.LinearDiscriminantAnalysis(n_components=2)
Wfld = fld.fit_transform(X, Y)

ifig = plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.scatter(Wpca[:,0], Wpca[:,1], c=Y, cmap=rbow, edgecolors='k')
plt.axis('equal'); plt.grid(True)
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title("PCA")
plt.subplot(1,2,2)
```

```
plt.scatter(Wfld[:,0], Wfld[:,1], c=Y, cmap=rbow, edgecolors='k')
plt.axis('equal'); plt.grid(True)
plt.xlabel('v1'); plt.ylabel('v2')
plt.title("FLD")
plt.close()
```

```
[53]: ifig
```

```
[53]:
```



```
[ ]:
```

30 Outline

1. Linear Dimensionality Reduction for Vectors
2. **Linear Dimensionality Reduction for Text**
3. Non-linear Dimensionality Reduction
4. Manifold Embedding

31 Dimensionality Reduction

- **Goal:** Transform high-dimensional vectors into low-dimensional vectors.
 - Dimensions in the low-dim data represent co-occurring features in high-dim data.
 - Dimensions in the low-dim data may have semantic meaning.
- **For example:** document analysis
 - high-dim: bag-of-word vectors of documents
 - low-dim: each dimension represents similarity to a topic.

32 Latent Semantic Analysis (LSA)

- Also called *Latent Semantic Indexing*
- Consider a bag-of-word representation (e.g., TF, TF-IDF)
 - document vector $\mathbf{x}_i$
 - $x_{i,j}$ is the frequency of word j in document i
- Approximate each document vector as a weighted sum of topic vectors.
 - $\hat{\mathbf{x}} = \sum_{n=1}^p w_p \mathbf{v}_p$
 - Topic vector $\mathbf{v}_p$ contains co-occurring words.
 - * corresponds to a particular *topic* or *theme*.
 - Weight w_p represents similarity of the document to the p -th topic.
- Objective:
 - minimize the squared reconstruction error (Similar to PCA):
 - $\min_{\mathbf{v}, \mathbf{w}} \sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$
 - Solution computed using SVD.
- Represent each document by its topic weights.
 - Apply other machine learning algorithms...
- **Advantage:**
 - Finds relations between terms (synonymy and polysemy).
 - distances/similarities are now comparing topics rather than words.
 - * higher-level semantic representation

33 Example on Spam Email dataset

- use bag-of-words representation with 50 words
- term-frequency (TF) normalization

```
[54]: # Load spam/ham text data from directories
textdata = datasets.load_files("email", encoding="utf8", decode_error="replace")

# convert to bag-of-words representation
cntvect = feature_extraction.text.CountVectorizer(stop_words='english',
↪max_features=50)
X = cntvect.fit_transform(textdata.data)
Y = textdata.target

# TF representation
tf_trans = feature_extraction.text.TfidfTransformer(norm='l1', use_idf=False)
Xtf = tf_trans.fit_transform(X)

# print the vocabulary
print(cntvect.vocabulary_)
```

```
{'online': np.int64(39), 'watches': np.int64(49), 'brands': np.int64(13),
'bags': np.int64(11), 'new': np.int64(35), 'today': np.int64(46), 'hi':
np.int64(25), 'peter': np.int64(42), 'contact': np.int64(17), 'mr':
np.int64(34), 'email': np.int64(21), 'com': np.int64(16), 'number':
```

```
np.int64(37), 'country': np.int64(19), 'nigeria': np.int64(36), 'answer':
np.int64(10), 'status': np.int64(44), 'good': np.int64(23), 'day': np.int64(20),
'john': np.int64(30), 'united': np.int64(47), 'payment': np.int64(40), 'bank':
np.int64(12), 'address': np.int64(9), 'information': np.int64(28), 'inform':
np.int64(27), '10': np.int64(2), 'office': np.int64(38), 'kamara': np.int64(32),
'000': np.int64(1), 'codeine': np.int64(15), '30mg': np.int64(6), '30':
np.int64(5), '00': np.int64(0), '60': np.int64(8), 'pills': np.int64(43), '120':
np.int64(3), 'just': np.int64(31), 'cost': np.int64(18), 'buy': np.int64(14),
'mg': np.int64(33), 'percocet': np.int64(41), '50': np.int64(7), '15mg':
np.int64(4), 'visa': np.int64(48), 'isidoro': np.int64(29), 'store':
np.int64(45), 'http': np.int64(26), 'google': np.int64(24), 'files':
np.int64(22)}
```

34 LSA on Spam data

- Apply LSA with 5 topics
 - implemented as TruncatedSVD

```
[55]: lsa = decomposition.TruncatedSVD(n_components=5, random_state=4487)
Wlsa = lsa.fit_transform(Xtf)

# components
V = lsa.components_
```

```
[56]: rbow = plt.get_cmap('rainbow')
def get_top_word_inds(T, vocab, mode='1std'):
    # return indices of the largest component entries
    absT = abs(T)

    # get 5 largest values
    if (mode=='top5'):
        tmp = argsort(absT)[::-1]
        topinds = tmp[0:5]

    elif (mode=='accum'):
        # select by accumulating
        tmpx = argsort(absT)[::-1]
        mags = absT[tmpx]
        magsc = cumsum(mags)
        magsc /= magsc[-1]
        #print magsc
        cumu = sum(magsc < 0.50)
        topinds = tmpx[0:cumu]

    elif ('1std'):
        # select words with magnitude of 1-std above mean
        mymn = mean(absT)
```

```

        mystd = std(absT)
        topinds = where(absT > mymn+mystd)[0]

# get the words
        topwords = []
        for q in topinds:
            mystr = vocab[q]
            if T[q]<0:
                mystr = "!" + mystr
            topwords.append(mystr)

        return topinds, topwords

def get_model(model):
    if model.__class__.__name__ == 'LatentDirichletAllocation':
        K = model.components_.shape[0]
        comps = model.components_.copy()
        # normalize to a probability distribution
        comps /= sum(comps,axis=1)[:,newaxis]
    else:
        K = model.n_components
        comps = model.components_
    return (K, comps)

def plot_topics(model, vocab, ind=None, figsize=(8,6), sp=None):
    # ind= topic indices to plot
    # figsize= figure size
    # sp= subplot indices (use None for default): [sy, sx, inds]
    K, comps = get_model(model)

    Xr = arange(model.components_.shape[1])
    lsafig = plt.figure(figsize=figsize)

    if ind is None:
        ind = range(K)
    if sp is None:
        sy = len(ind)
        sx = 1
        spinds = range(1,sy+1)
    else:
        sy = sp[0]
        sx = sp[1]
        spinds = sp[2:]

    for ip,j in enumerate(ind):
        plt.subplot(sy,sx,spinds[ip])
        # get components

```

```

T = comps[j,:]

tmp,topwords = get_top_word_inds(T, vocab)

plt.bar(Xr, T, color='none', edgecolor='k')
plt.plot(tmp, T[tmp], 'bo')
# plot words on the top 5
TW = []
for i,q in enumerate(tmp):
    mystr = topwords[i]
    # plt.annotate(xy=(q+0.4,T[q]), text=mystr,
    # xytext=(q+0.4,T[q]), rotation=45,
    # ha='left', va='bottom', color='r')
    plt.annotate(xy=(q+0.4,T[q]-0.05), text=mystr,
                 xytext=(q+0.4,T[q]-0.05),
                 ha='left', va='bottom', color='r')

plt.grid(True)
plt.ylabel('Topic ' + str(j+1))
if ip == sy-1:
    plt.xlabel('words')
else:
    plt.gca().set_xticklabels([])
plt.xlim(Xr[0]-0.5, Xr[-1]+0.5)
if sp is None:
    # plt.tight_layout()
    plt.close()
return lsafig

```

35 Topic vectors

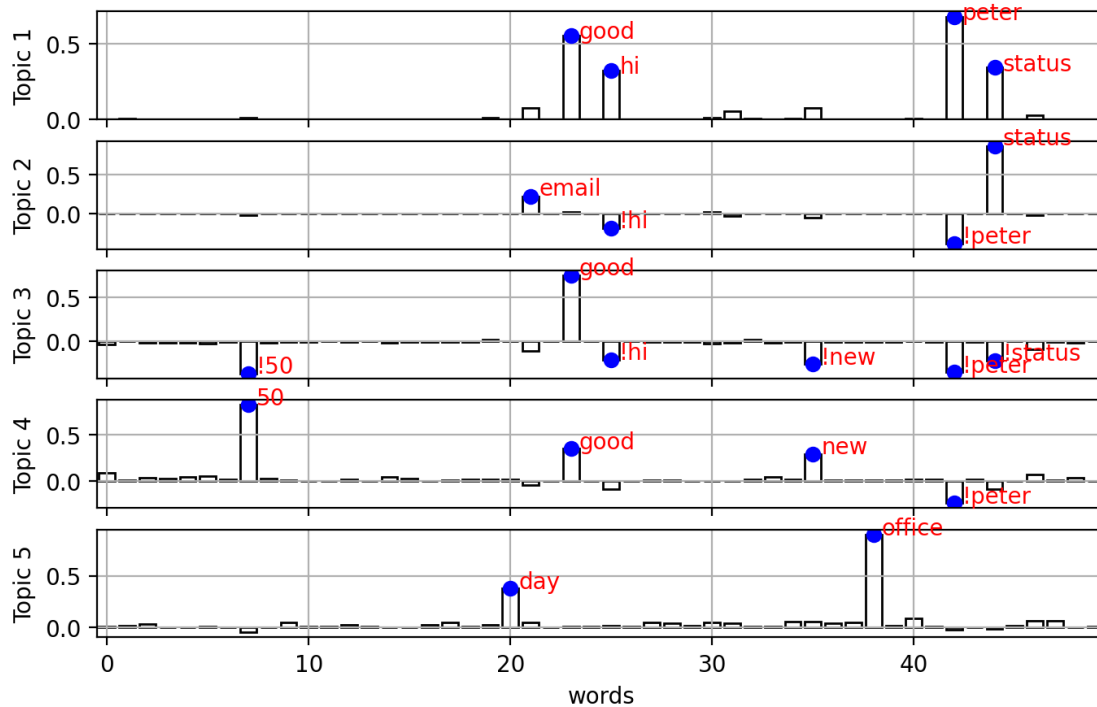
- topic vectors contain frequent co-occurring words

```

[57]: vocab = asarray(cntvect.get_feature_names_out())
lsafig = plot_topics(lsa, vocab, figsize=(8,5))
lsafig

```

[57]:



36 Document representation

- Documents are a combination of topics

```
[58]: def plot_doc_topic(myXtf, myW, model, vocab):
    docfig = plt.figure(figsize=(8,6))
    plt.subplot(2,1,1)
    K, comps = get_model(model)
    Xr = arange(comps.shape[1])
    myXtfa = myXtf.toarray().flatten()
    plt.bar(Xr, myXtfa, lw=1, color="none", label='document', edgecolor='k')
    plt.bar(Xr, myXtfa, lw=0, alpha=0.15, color='k')
    for j in range(K):
        T = comps[j,:]
        mystr = "Topic " + str(j+1) + " ($w_{"+str(j+1)+"}$=% .4f"%(myW[j],) + ")_
        ↪

        # get largest values, and add words
        tmp, topwords = get_top_word_inds(T, vocab)
        mystr += " ".join(topwords)
        plt.bar(Xr, myW[j]*T, color="none", edgecolor=rbow(float(j)/K), lw=2,
        ↪label=mystr)
```

```

plt.grid(True); plt.xlabel('words')
tmpax = plt.axis()
plt.axis([Xr[0]-0.5, Xr[-1]+0.5, tmpax[2], 1.0])
plt.legend(fontsize=8, loc='upper center')

plt.subplot(2,1,2)
plt.bar(Xr, myXtfa, lw=1, color="none", label='original document',
edgecolor='k')
plt.bar(Xr, myXtfa, lw=0, alpha=0.15, color='k')
newX = dot(myW[newaxis,:],comps).flatten()
plt.bar(Xr, newX, color="none", edgecolor='r', lw=2, label='reconstruction')

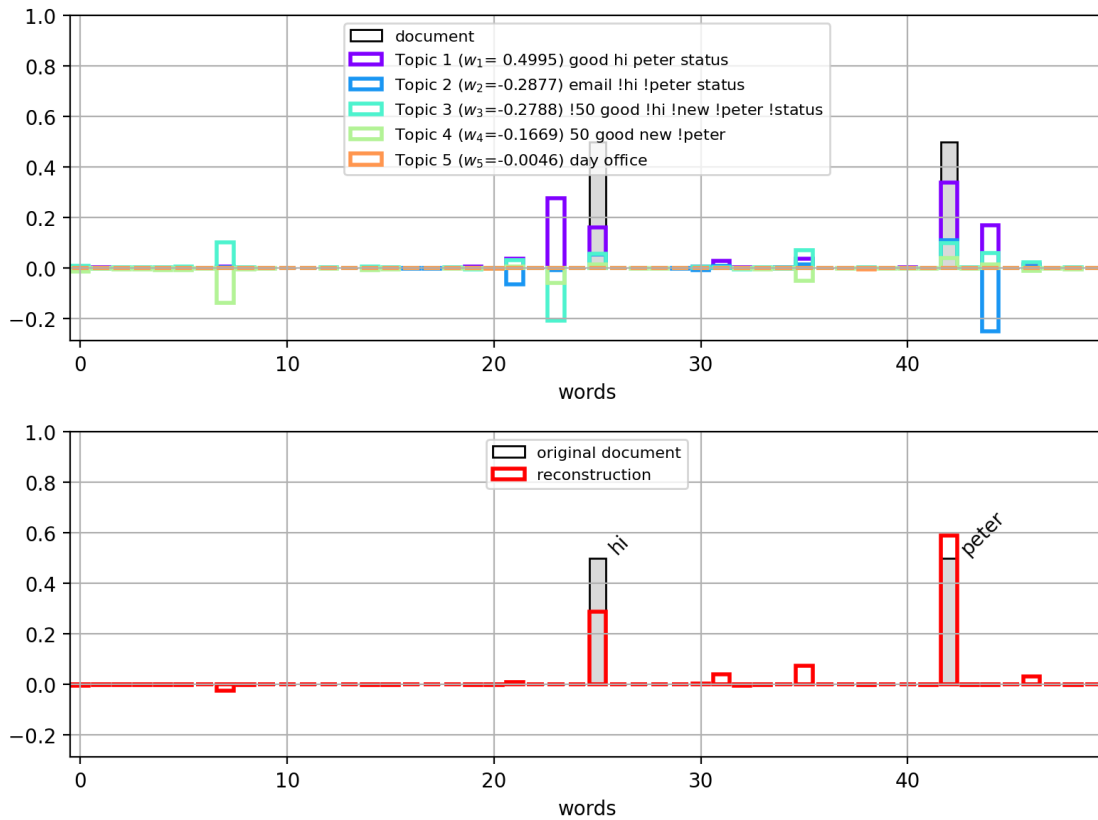
# get largest values
cols = ('k', 'r')
for i,T in enumerate([myXtfa, newX]):
    if i==0:
        tmp = where(abs(T)>=0.1)[0]
    else:
        tmp = where((abs(T)>=0.1) & (abs(myXtfa)<0.1))[0]
    #plt.plot(tmp+0.4, T[tmp], 'ko')
    for q in tmp:
        mystr = vocab[q]
        if T[q]<0:
            mystr = "!" + mystr
        plt.annotate(xy=(q+0.4,T[q]), text=mystr,
                    xytext=(q+0.4,T[q]), rotation=45,
                    ha='left', va='bottom', color=cols[i])

plt.grid(True)
plt.xlabel('words')
plt.axis([Xr[0]-0.5, Xr[-1]+0.5, tmpax[2], 1.0])
plt.legend(fontsize=8, loc='upper center')
plt.tight_layout()
plt.close()
return docfig

```

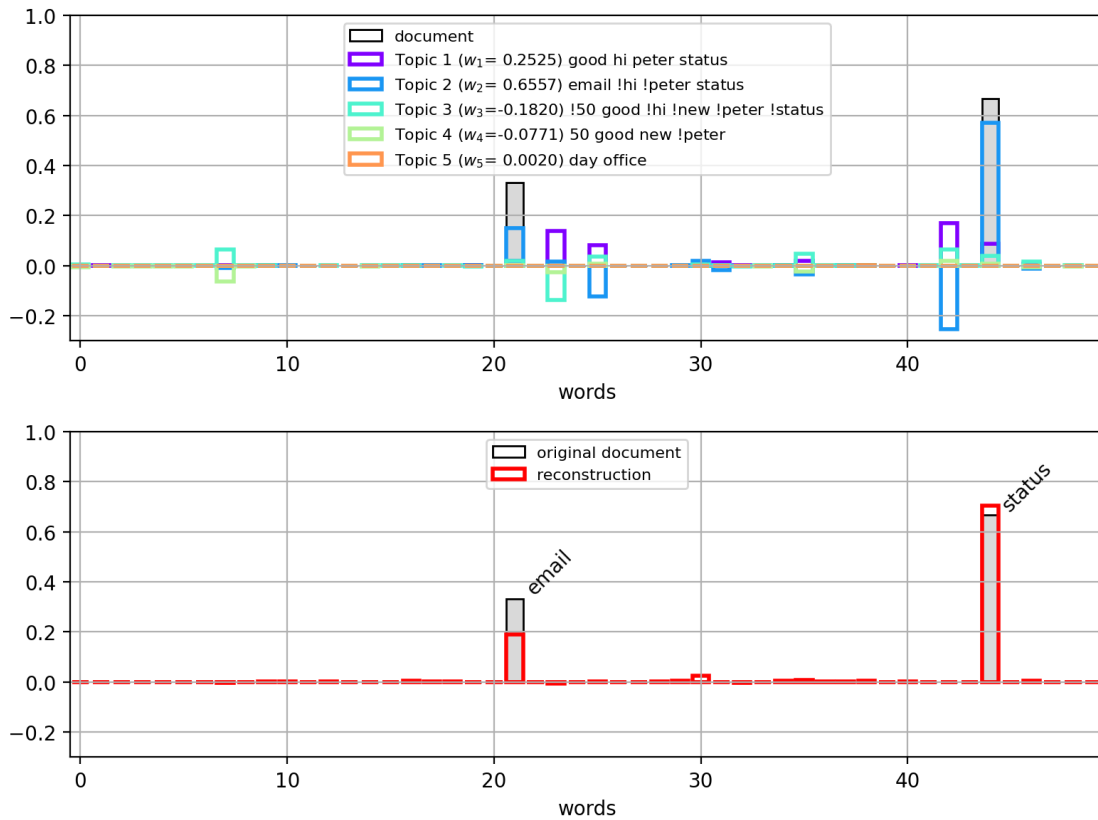
```
[59]: plot_doc_topic(Xtf[2,:], Wlsa[2,:], lsa, vocab)
```

```
[59]:
```



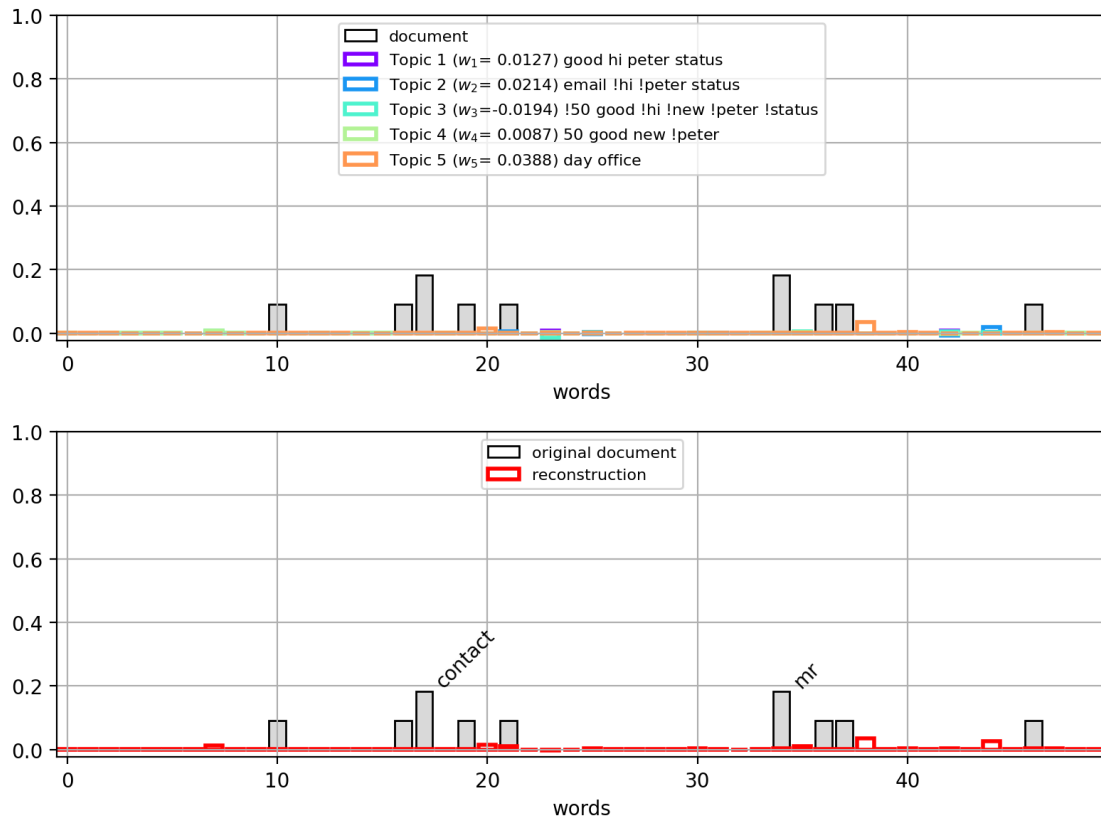
```
[60]: plot_doc_topic(Xtf[4,:], Wlsa[4,:], lsa, vocab)
```

```
[60]:
```



```
[61]: plot_doc_topic(Xtf[3,:], Wlsa[3,:], lsa, vocab)
```

```
[61]:
```

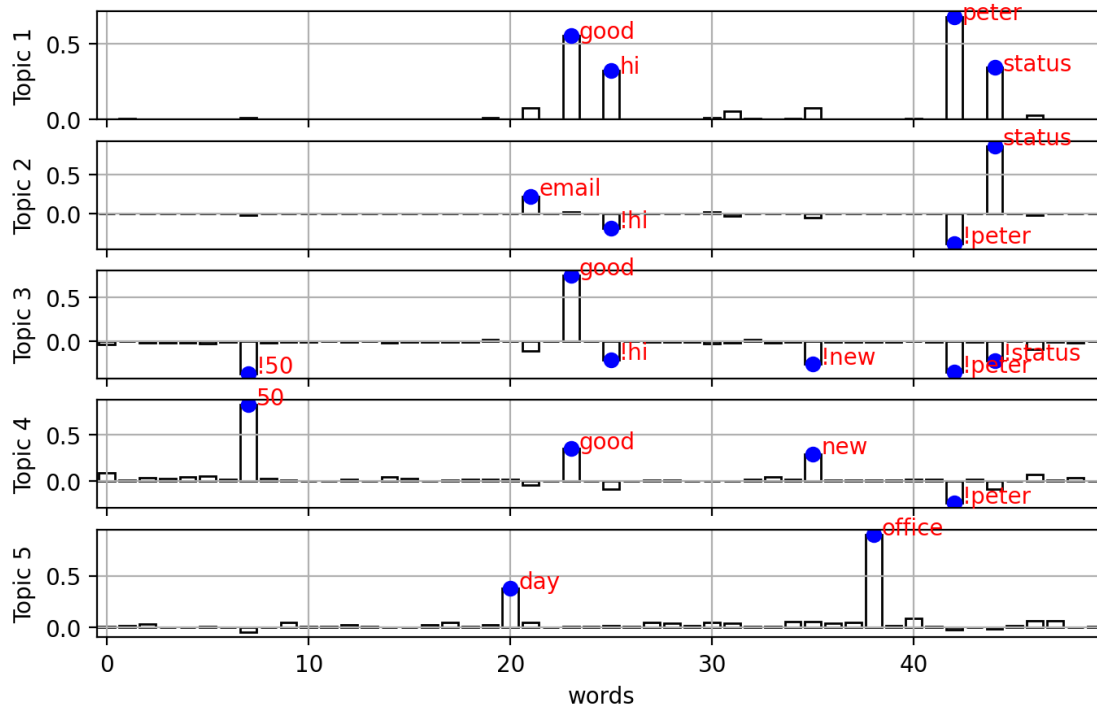


37 Problem with LSA

- In the topic vector, the “frequency” of a word can be negative!
 - Doesn’t really make sense for document bag-of-words model.

[62]: lsafig

[62]:



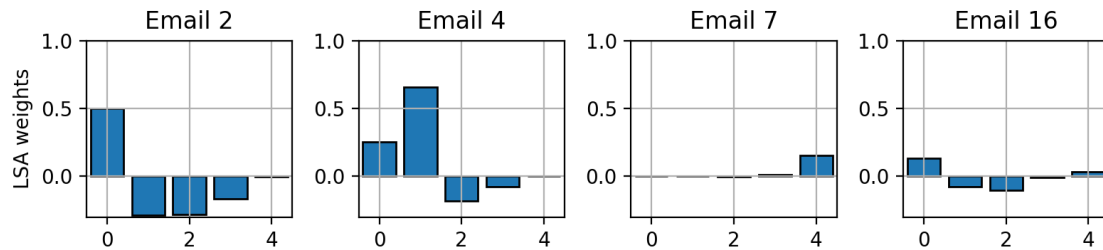
38 Problems with LSA

- The weights for each topic can be negative!
 - Topics should only be “additive”
 - * Topics should increase probability of some topic-related words, but not decrease probability of other words.
 - * It doesn’t make sense to “remove” a topic using a negative topic weight.

```
[63]: pfig = plt.figure(figsize=(8,2))
for i,e in enumerate([2, 4, 7, 16]):
    plt.subplot(1,4,i+1)
    plt.bar(range(5), Wlsa[e,:], edgecolor='k')
    plt.title('Email ' + str(e))
    if i==0:
        plt.ylabel('LSA weights')
    plt.grid(True)
    plt.axis([-0.5, 4.5, -0.3, 1])
plt.tight_layout()
plt.close()
```

```
[64]: pfig
```

```
[64]:
```



39 Non-negative Matrix Factorization (NMF)

- **Solution:** constrain the topic vector and weights to be non-negative.
- Similar to LSA
 - Approximate each document vector as a weighted sum of topic vectors.
 - * $\hat{\mathbf{x}}_j = \sum_{n=1}^p w_p \mathbf{v}_p$
 - * But now, each entry of topic vector $\mathbf{v}_p \geq 0$ and topic weight $w_p \geq 0$
 - Objective: minimize the squared reconstruction error

$$\min_{\mathbf{v} \geq 0, \mathbf{w} \geq 0} \sum_j \|\mathbf{x}_j - \hat{\mathbf{x}}_j\|^2$$

- * subject to the non-negative constraints.
- * no closed-form solution, need to use an optimizer.

```
[65]: # Run NMF
nmf = decomposition.NMF(n_components=5)
Wnmf = nmf.fit_transform(Xtf)

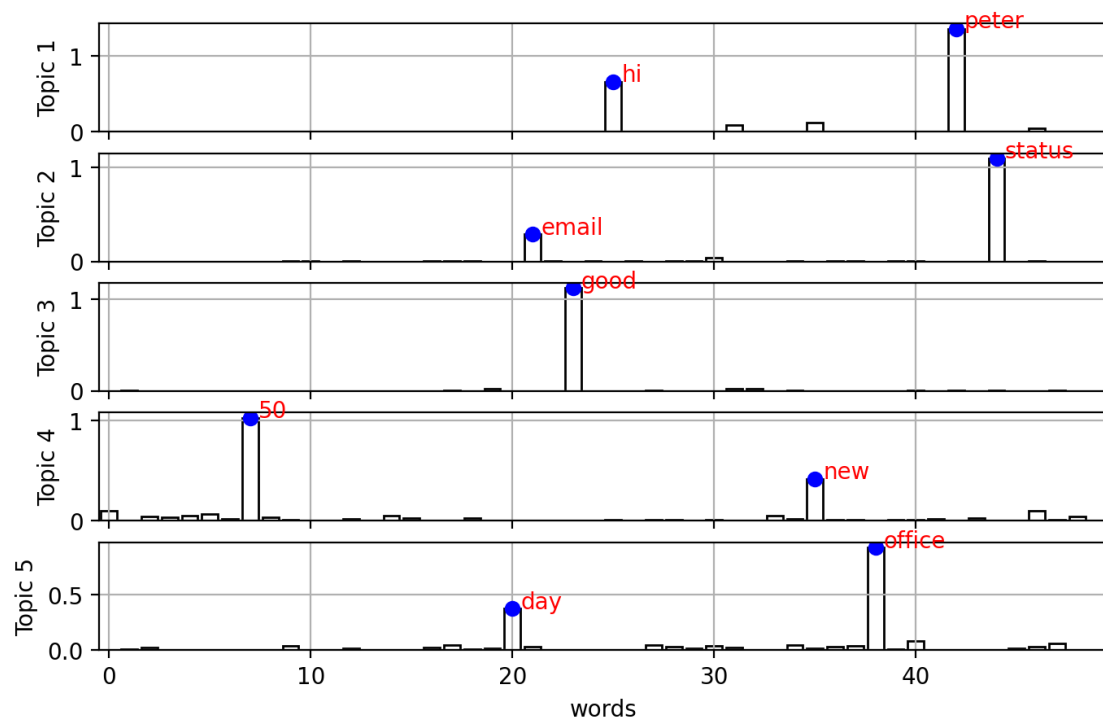
# components
V = nmf.components_
```

40 Topic vector

- all non-negative entries
- looks much cleaner (less small entries)

```
[66]: plot_topics(nmf, vocab, figsize=(8,5))
```

```
[66]:
```

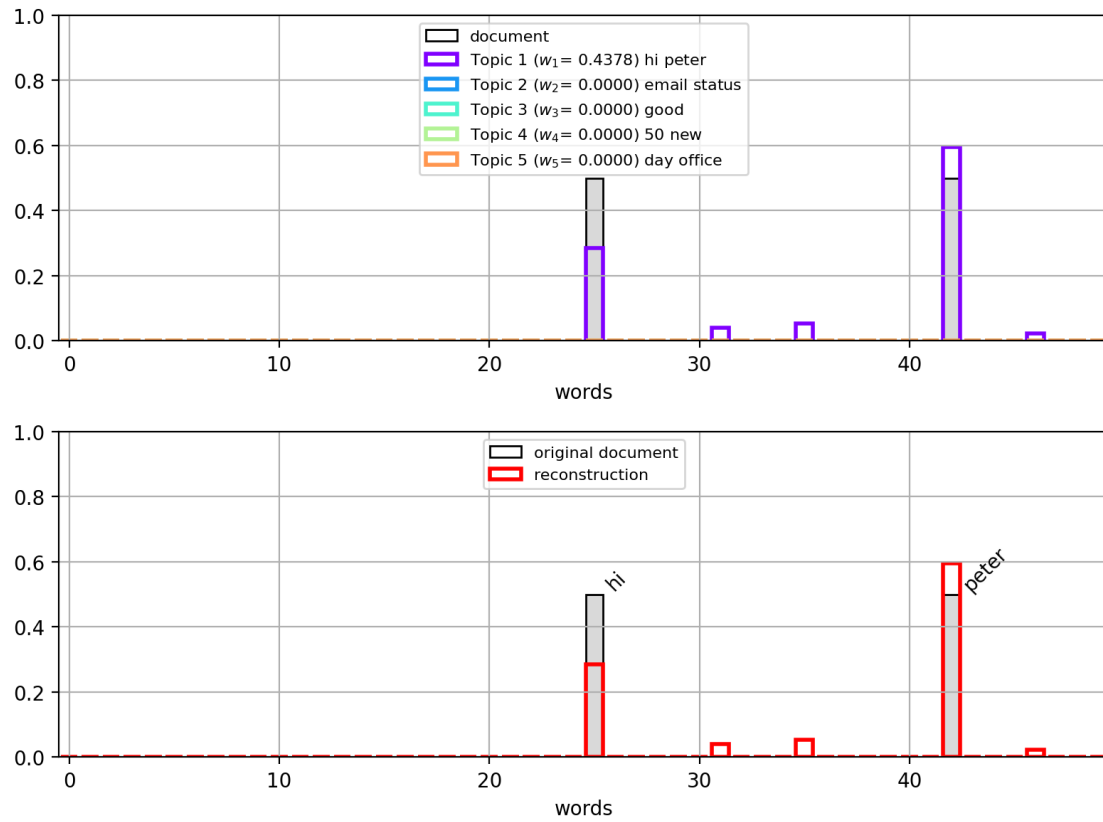


41 Document vector

- additive combination of topics

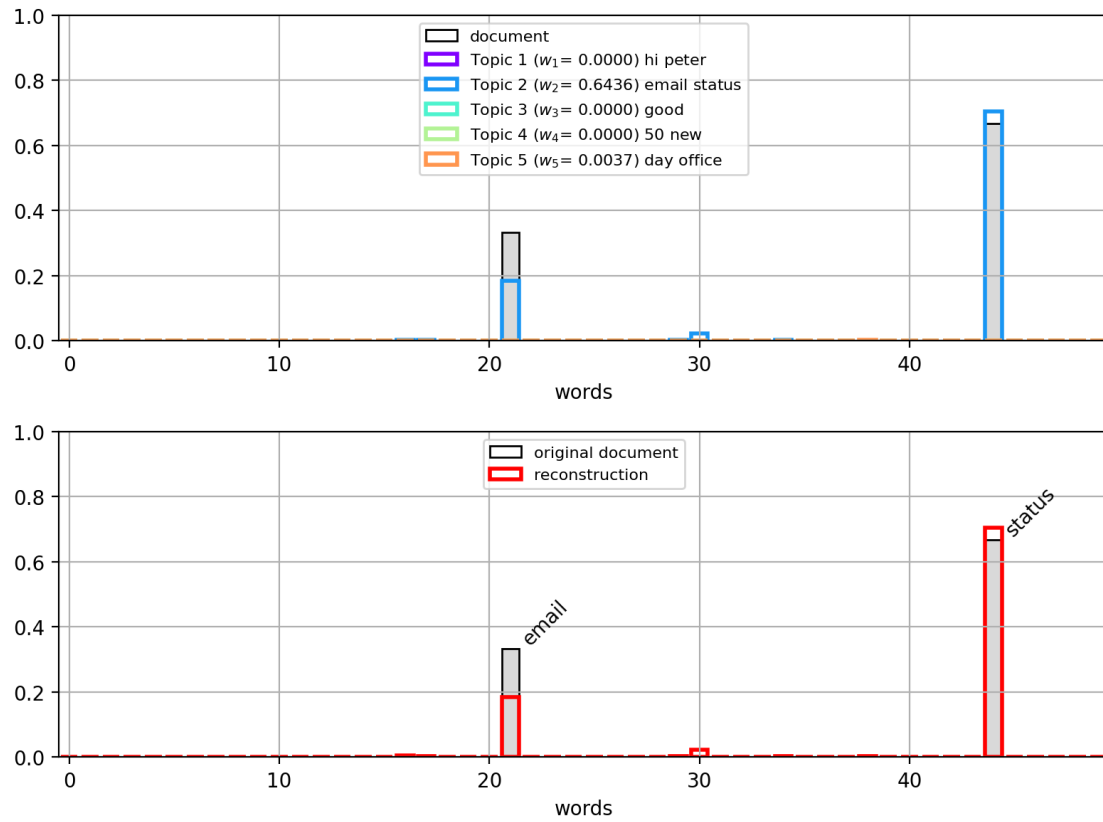
```
[67]: plot_doc_topic(Xtf[2,:], Wnmf[2,:], nmf, vocab)
```

```
[67]:
```

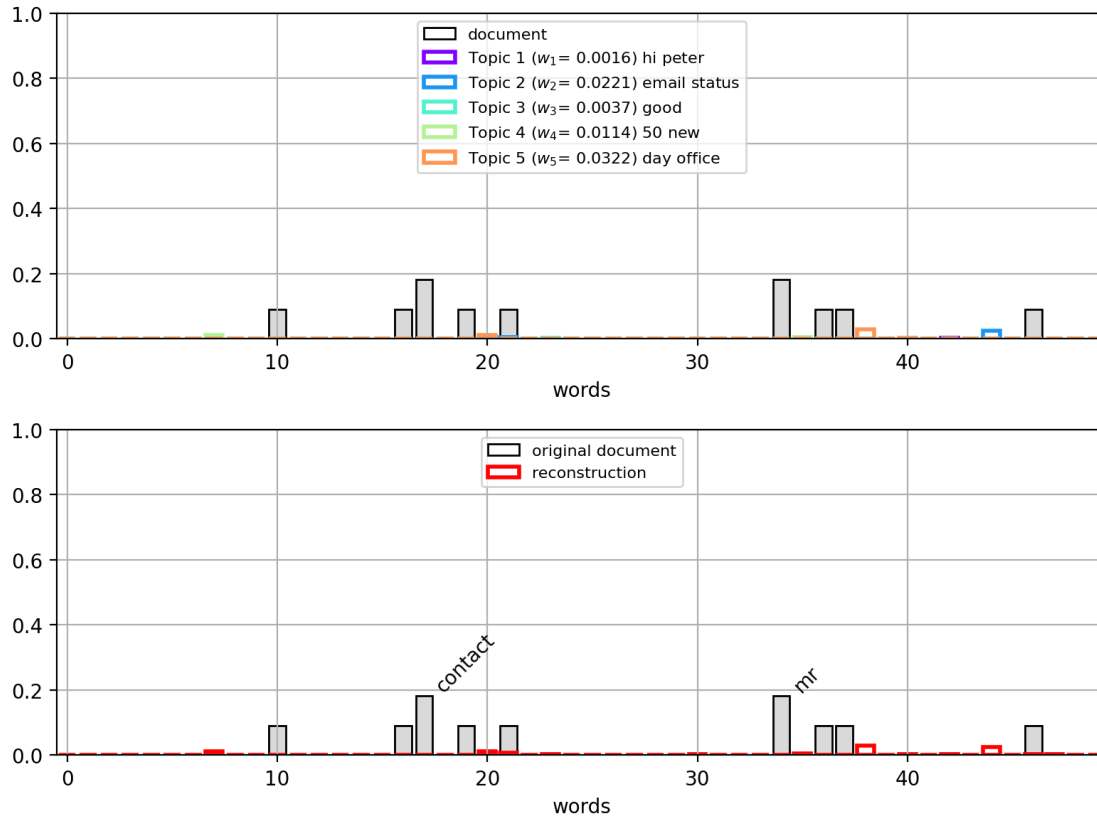
```
[68]: plot_doc_topic(Xtf[4,:], Wnmf[4,:], nmf, vocab)
```

```
[68]:
```



```
[69]: plot_doc_topic(Xtf[3,:], Wnmf[3,:], nmf, vocab)
```

```
[69]:
```



42 Sparseness

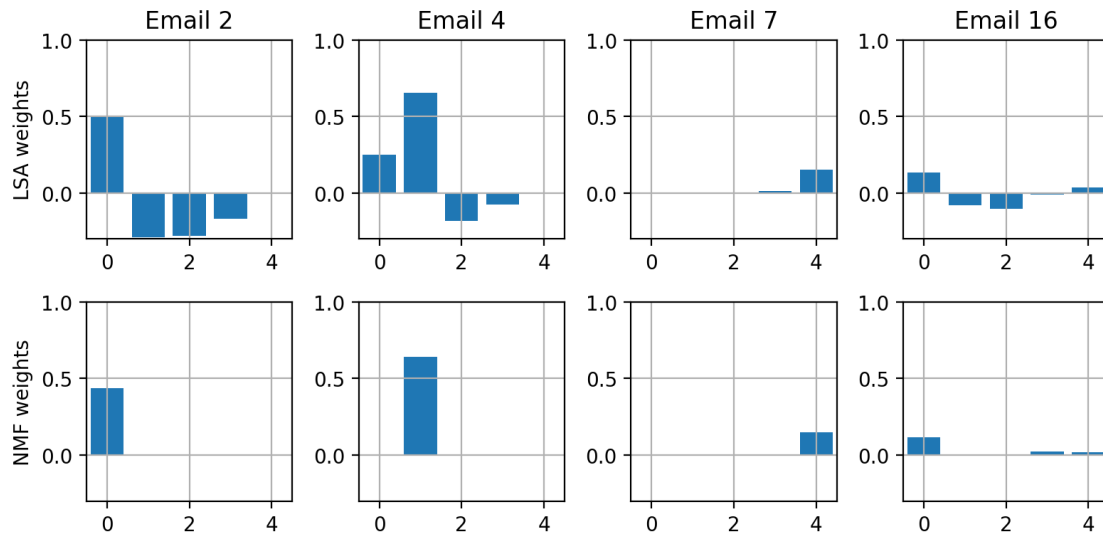
- For NMF representation, most topic weights for a document are zero.
 - this is called a *sparse* representation.
 - each document is only composed of a few topics.

```
[70]: spfig = plt.figure(figsize=(8,4))
for i,e in enumerate([2, 4, 7, 16]):
    plt.subplot(2,4,i+1)
    plt.bar(range(5), Wlsa[e,:])
    plt.title('Email ' + str(e))
    if i==0:
        plt.ylabel('LSA weights')
    plt.grid(True)
    plt.axis([-0.5,4.5,-0.3,1])
    plt.subplot(2,4,i+1+4)
    plt.bar(range(5), Wnmf[e,:])
    plt.axis([-0.5,4.5,-0.3,1])
    if i==0:
        plt.ylabel('NMF weights')
```

```
plt.grid(True)
plt.tight_layout()
plt.close()
```

```
[71]: spfig
```

```
[71]:
```



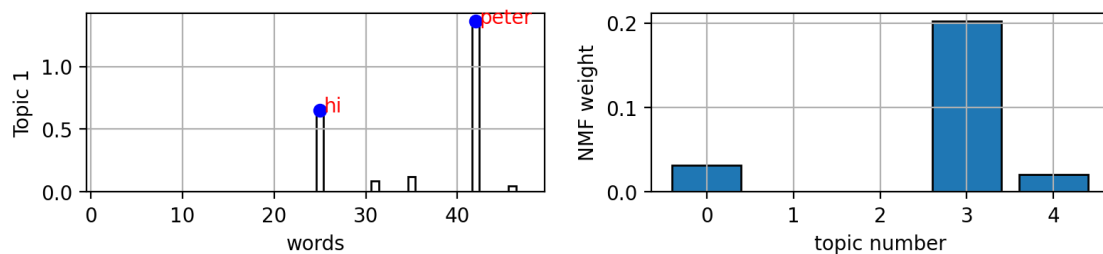
43 Problem with NMF

- While the weights and component vectors are non-negative, NMF does not enforce them to be probabilities.

```
[72]: nfig = plot_topics(nmf, vocab, [0], figsize=(8,2), sp=(1,2,1))
plt.subplot(1,2,2)
plt.bar(range(5), Wnmf[1,:], edgecolor='k');
plt.xlabel('topic number'); plt.ylabel('NMF weight')
plt.grid(True)
plt.tight_layout()
plt.close()
```

```
[73]: nfig
```

```
[73]:
```



- TF/TFIDF is a probabilistic model of words in a document
 - the vector of probabilities sums to 1
 - probabilities are between 0 and 1
- The NMF components and weights are difficult to interpret.
- *We will solve this problem later using probabilistic graphical models.*

44 Linear Dimensionality Reduction - Summary

- **Goal:** given set of input vectors $\{\mathbf{x}_i\}_{i=1}^n$, with $\mathbf{x}_i \in \mathbb{R}^d$, represent each input vector as lower-dimensional vector $\mathbf{w}_i \in \mathbb{R}^p$.
 - Approximate $\mathbf{x}$ as a weighted sum of basis vectors $\mathbf{v}_j \in \mathbb{R}^d$
 - * $\hat{\mathbf{x}} = \sum_{j=1}^p w_j \mathbf{v}_j$
 - * minimize the reconstruction error of $\hat{\mathbf{x}}$.
 - enables faster processing, or reduces noise.

45 Other things

- *Feature Normalization*
 - PCA and LDA are based on the covariance between input dimensions.
 - applying *per-feature* normalization will yield a different PCA result!
 - * normalizing each input dimension changes the relative covariances.

```
[74]: def plot_basis(X, vo, Xp=None, sp=None, showleg=True, Y=None, axbox1=None,
↪axbox2=None):
    if sp is None:
        sp = ((1,2,1), (1,2,2))
    cols = ['k', 'g']

    axh = [None, None]

    if Y is None:
        Y = zeros(len(X))
        mycmap = plt.get_cmap('bwr')
    else:
        mycmap = rbow

    # normalize v
    v = vo.copy()
    numd = v.shape[0]
    for j in range(numd):
        v[j,:] /= linalg.norm(v[j,:])
        # flip direction if necessary
        if all(v[j,:] < 0):
            v[j,:] *= -1.0
```

```

Xm = mean(X,axis=0)           # get mean
if Xp is None:
    Xp = dot(X-Xm,v.T) # project data onto v

axh[0] = plt.subplot(sp[0][0], sp[0][1], sp[0][2])
# plot points
plt.scatter(X[:,0], X[:,1], s=5, c=Y, cmap=mycmap, edgecolors='k')
if axbox1 is not None:
    plt.axis(axbox1)
else:
    plt.axis('equal')
plt.grid(True)
plt.title('Original space (x)')

tmp = plt.xlim()
hw = (tmp[1]-tmp[0])/30
lw = hw/10

mind = zeros(numd)
maxd = zeros(numd)
Ls = []
LL = []
for j in range(numd):
    mind[j] = Xp[:,j].min()
    maxd[j] = Xp[:,j].max()

    ofact = 0.9
    Xm2 = Xm + v[j]*maxd[j]*ofact
    Xm1 = Xm + v[j]*mind[j]*ofact
    vl = v[j]*(maxd[j]-mind[j])*ofact

    tmp = plt.arrow(Xm1[0], Xm1[1], vl[0], vl[1], width=lw, head_width=hw,
                    color=cols[j], length_includes_head=True)
    Ls.append(tmp)
    LL.append("PC "+str(j))

plt.plot(Xm[0], Xm[1], 'ks')
if showleg:
    plt.legend(Ls, LL, loc='lower right')

axh[1] = plt.subplot(sp[1][0], sp[1][1], sp[1][2])
plt.arrow(mind[0]*ofact, 0, (maxd[0]-mind[0])*ofact, 0, width=lw,
↪head_width=hw, color=cols[0])
if numd == 2:
    plt.arrow(0, mind[1]*ofact, 0, (maxd[1]-mind[1])*ofact, width=lw,
↪head_width=hw, color=cols[1])

```

```

        plt.scatter(Xp[:,0], Xp[:,1], s=10, marker='+', c=Y, cmap=mycmap,
↪edgecolors='k')
        plt.xlabel('w1')
        plt.ylabel('w2')
    else:
        plt.scatter(Xp[:,0], zeros(len(Xp)), s=10, marker='x', c=Y, cmap=mycmap)
        #, edgecolors='k')
        plt.xlabel('w')

    if axbox2 is not None:
        plt.axis(axbox2)
    else:
        plt.axis('equal')
    plt.grid(True)
    plt.title('Low-dim space (w)')

    return axh

```

```

[75]: iris = datasets.load_iris()
X = iris.data[:,(2,1)]
Y = iris.target

# normalize the data
scaler = preprocessing.StandardScaler()
Xs = scaler.fit_transform(X)

# PCA on original
pca = decomposition.PCA(n_components=1)
Wpca = pca.fit_transform(X)

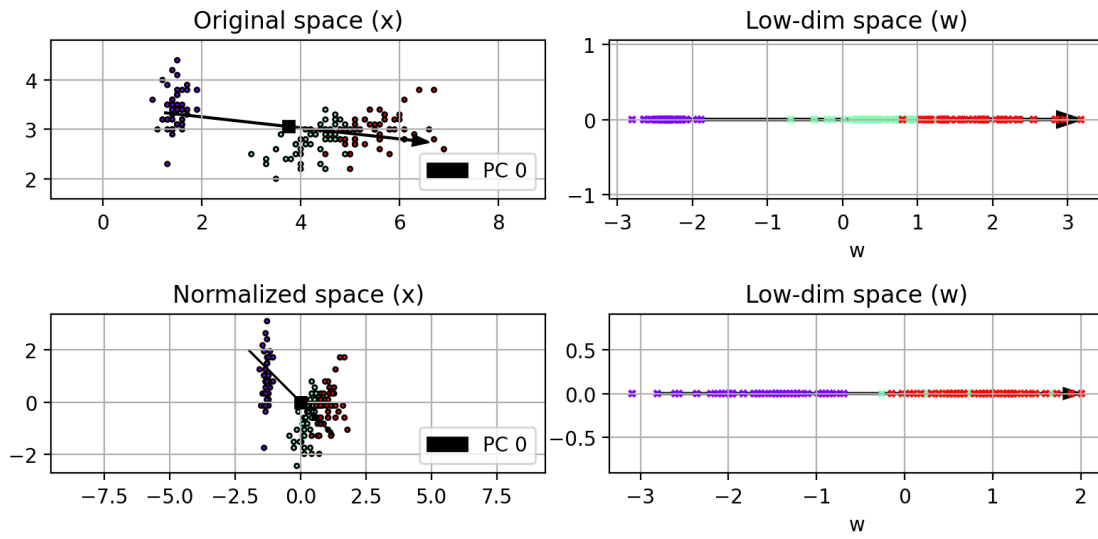
# PCA on normalized
pca2 = decomposition.PCA(n_components=1)
Wpca2 = pca2.fit_transform(Xs)

nfig = plt.figure(figsize=(8,4))
axh1 = plot_basis(X, pca.components_, sp=((2,2,1),(2,2,2)), Y=Y)
axh2 = plot_basis(Xs, pca2.components_, sp=((2,2,3),(2,2,4)), Y=Y)
plt.sca(axh2[0])
plt.title('Normalized space (x)')
plt.tight_layout()
plt.close()

```

```
[76]: nfig
```

```
[76]:
```



[]: